

Git 与 VS Code 实战教程

从入门到精通的完整指南

涵盖 Git 核心概念、GitHub 协作与 VS Code 深度集成

全彩插图版 • 含 27 张 VS Code 真实界面截图

2026 年 8 月 3 日

关于本文档

本文档是一份面向初学者的 Git 与 GitHub 完整教程，重点讲解如何在 Visual Studio Code（VS Code）中利用 Git 进行高效的代码版本管理。

本版本特色：

- 全文包含 **27 张** VS Code 真实界面截图，每个操作都有对应的界面演示
- 从概念到实操，图文并茂，降低学习门槛
- 同时覆盖命令行和 VS Code 图形界面两种操作方式

适用对象：编程初学者、希望学习版本控制的开发者、希望从命令行迁移到图形化工作流的 Git 用户。

前置要求：基本的计算机操作能力；已安装或准备安装 Git 和 VS Code。

目录

第 0 章：开始前的准备——建立心智模型	7
0.1 三个核心类比	7
0.1.1 Git 是「时光机」	8
0.1.2 GitHub 是「云端仓库」	9
0.1.3 VS Code 是「驾驶舱」	9
0.2 Git 的设计哲学：四个关键词	9
0.2.1 关键词一：每个存档都有独一无二的编号	10
0.2.2 关键词二：保存完整快照，不是只记差异	10
0.2.3 关键词三：历史是一张网，不是一条直线	10
0.2.4 关键词四：分支和标签只是小纸条	10
0.3 本教程的学习路径图	10
0.4 动手实验 0：验证环境就绪	11
第一章 版本控制与 Git 概述	12
1.1 什么是版本控制	12
1.2 Git 简介	14
1.2.1 Git 的核心特性	14
1.3 Git 的内部数据模型	15
1.3.1 Git 的四个核心对象	15
1.3.2 HEAD 与引用	16
1.3.3 一次提交的完整结构	17
1.4 GitHub 简介	18
1.5 Git 的三大工作区域	19
1.6 VS Code 如何与 Git 协作	20
第二章 Git 安装与配置	22
2.1 安装 Git	22
2.1.1 Windows 系统	22

2.1.2	macOS 系统	23
2.1.3	Linux 系统	23
2.2	基本配置	23
2.2.1	用户身份（必须配置）	24
2.2.2	其他推荐配置	24
2.2.3	配置层级	25
2.3	安装 VS Code	25
2.4	配置 GitHub 认证	26
2.4.1	方式一：GitHub CLI（推荐）	26
2.4.2	方式二：Personal Access Token	26
2.4.3	方式三：SSH 密钥（最推荐）	27
第三章	Git 基础操作	29
3.1	初始化仓库	29
3.1.1	方式一：在已有项目中初始化	30
3.1.2	方式二：克隆远程仓库	30
3.2	文件状态与跟踪	31
3.3	日常工作流：status → add → commit	32
3.3.1	第一步：查看状态（git status）	32
3.3.2	第二步：暂存文件（git add）	33
3.3.3	第三步：提交变更（git commit）	34
3.3.4	编写好的提交信息	35
3.4	查看修改内容：Diff 视图	35
3.4.1	打开 Diff 视图	36
3.4.2	并排视图 vs 内联视图	36
3.4.3	编辑器中的实时变更指示	37
3.4.4	命令行 diff 详解	37
3.5	查看历史：git log	38
3.6	撤销操作	39
3.6.1	撤销工作区的修改（未 add）	39
3.6.2	取消暂存（已 add，未 commit）	39
3.6.3	修改上一次提交	40
3.6.4	回退到之前的版本	40
3.7	.gitignore 文件	41
第四章	分支与合并	44
4.1	分支的概念	44
4.2	分支操作	45

4.2.1	创建、切换、删除分支	45
4.2.2	分支命名规范	47
4.3	合并分支	47
4.3.1	合并的两种类型	48
4.4	解决合并冲突	48
4.4.1	冲突标记	48
4.4.2	VS Code 的冲突解决体验	49
4.4.3	解决冲突的完整步骤	50
4.4.4	远程协作中的冲突（PR 冲突）	51
4.5	Rebase: 整理历史	53
4.5.1	Rebase 与 Merge 的区别	53
4.5.2	Rebase 的使用场景	54
4.6	Cherry-pick: 挑选提交	55
4.7	分支策略简介	55
4.7.1	Feature Branch 工作流（推荐）	55
4.7.2	GitFlow 工作流	56
第五章	远程仓库与 GitHub	58
5.1	远程仓库的概念	58
5.2	推送与拉取	59
5.2.1	git push: 上传本地提交到远程	59
5.2.2	git fetch: 下载远程更新（不合并）	59
5.2.3	git pull: 下载并合并远程更新	60
5.3	在 GitHub 上创建仓库	61
5.3.1	方式一: 先在 GitHub 创建, 再克隆到本地	61
5.3.2	方式二: 先在本地创建, 再推送到 GitHub	61
5.3.3	方式三: VS Code 一键发布	62
5.4	Pull Request (PR)	62
5.4.1	PR 的标准流程	63
5.4.2	PR 工作流程图	64
5.5	Fork 与贡献开源项目	64
第六章	VS Code 中的 Git 集成	66
6.1	源代码管理视图 (Source Control)	66
6.1.1	打开方式	66
6.1.2	界面详解	67
6.1.3	提交历史图 (Source Control Graph)	68
6.2	暂存 (Stage) 与提交 (Commit) 详解	68

6.2.1	暂存文件	68
6.2.2	部分暂存 (Stage Selected Ranges)	68
6.3	同步操作详解	69
6.3.1	一键同步	69
6.4	分支管理详解	70
6.4.1	查看和切换分支	70
6.5	Timeline 视图: 文件历史	70
6.6	Stash (暂存工作)	71
6.7	Git Blame 与行内注释	71
6.8	推荐扩展	72
6.8.1	GitLens 详细使用指南	73
6.8.2	GitHub Pull Requests 使用指南	74
6.8.3	GitHub Issues 使用指南	75
6.8.4	Git Graph 使用指南	76
6.8.5	Git History 使用指南	76
第七章	实战工作流	79
7.1	完整的 Feature Branch 开发流程	79
7.2	VS Code 中的完整操作流程	81
7.3	日常开发的最佳实践	81
7.4	常见问题 FAQ	82
第八章	新手避坑指南	85
8.1	核心避坑表	85
8.2	紧急救命命令速查	86
8.3	常见报错与一键修复	87
可视化速查卡		88
8.4	速查卡 1: Git 四大区域数据流转	88
8.5	速查卡 2: 分支演变——合并 vs 变基	89
8.6	速查卡 3: 远程协作完整链路	90
第九章	附录: 常用命令速查表	91
9.1	基础操作	91
9.2	分支操作	91
9.3	远程操作	92
9.4	撤销操作	92
9.5	VS Code 快捷键速查	93

第 0 章：开始前的准备——建立心智模型

如果你完全没接触过 Git、GitHub、版本控制，请先花 5 分钟读完这一章。不需要记命令，只要建立三个 ** 生活类比 **，后面学起来就会顺畅很多。

⚠ 注意

为什么需要版本控制？

想象一下这样的场景：你正在写一个网站项目，代码文件名叫 `index.html`。你改了一版，觉得不太对，于是复制了一份，命名为 `index_v2.html`；又改了一版，变成 `index_v3_final.html`；后来又发现有个 bug，于是有了 `index_v3_final_真的.html`……一周以后，你的文件夹里堆满了各种版本的文件，你已经完全忘记哪个才是“最终版”。

这还只是一个人工作的场景。如果团队里三个人同时修改同一个文件呢？你改完传给同事，同事改完传回来，你再改——文件名变成了 `index_张三_改.html`、`index_李四_改_最终.html`。最后，你们都不知道该用哪个版本了。

版本控制系统就是为解决这些问题而生的。它自动记录每一次修改，让你可以随时回到过去的任意版本，还能让多人协作时不再“文件满天飞”。

0.1 三个核心类比

Git、GitHub 和 VS Code 这三个名字经常一起出现，但它们各自扮演的角色其实截然不同。对初学者来说，最大的困惑往往是“到底该用哪个工具做什么”。直接记命令容易忘，不如先在大脑里建立三个鲜活的画面。我们把 Git 想象成一台时光机，它负责记录你代码的每一次变化，让你随时能回到过去。GitHub 则像一座放在云端的仓库，专门帮你把代码备份到网上，同时让多人协作变得简单。VS Code 是驾驶舱，它把 Git 和 GitHub 的能力包装成按钮和菜单，让你不用记命令也能完成操作。三者关系如图 1 所示——图中三条箭头说清了它们的分工：Git 负责保存历史，GitHub 负责云端同步与协作，VS Code 负责把这一切翻译成图形界面，让你点按钮就能完成操作。

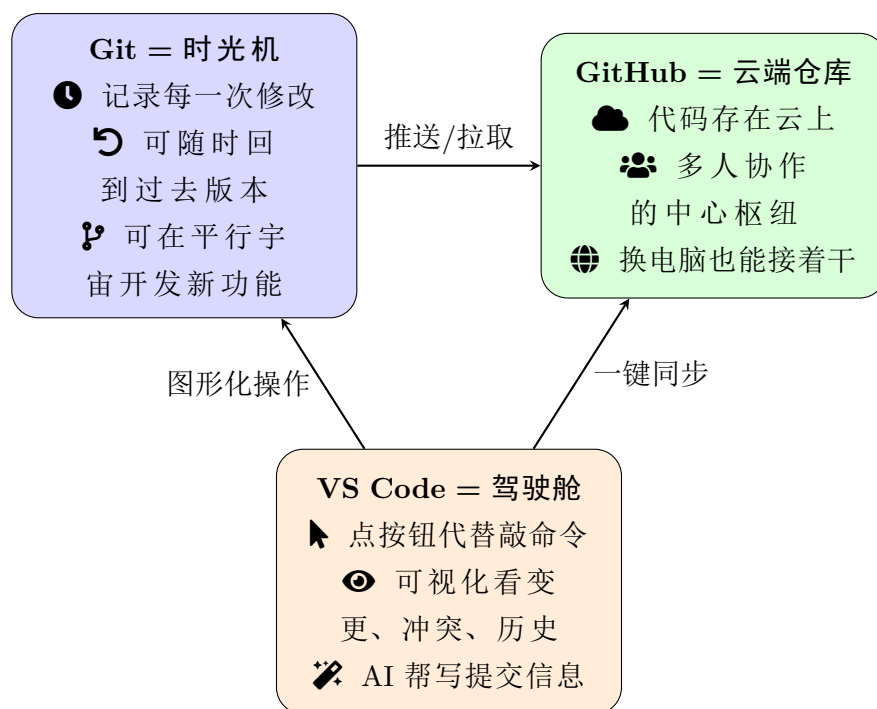


图 1: Git、GitHub 与 VS Code 的关系——三者协作构成完整的版本控制 workflow

0.1.1 Git 是「时光机」

玩过单机游戏的人对“存档”一定不陌生。平时随便存档不心疼，关键 boss 战前存一个，打崩了就读档重来。Git 的每次提交（commit）就像给整个项目拍了一张照片，照片会被完整保存下来。想回到过去任何一刻都可以，而且“读档”不会弄坏你当前的工作，因为 Git 的读档不会动到你当前的工作区。如果你突然想尝试一个新想法，可以开一条新的剧情线，这就是分支，几条线互不影响，搞砸了删掉重来就行。

- **写代码 = 穿越时空**：每次 `git commit` 就是按下「存档」，生成一个唯一的「存档编号」（哈希值）。
- **想回去 = 时光倒流**：`git restore` / `git reset` 让你回到任意一次存档的瞬间。
- **想尝试新想法 = 进入平行宇宙**：`git switch -c` 新分支创建一个互不干扰的平行世界，搞砸了删掉分支就行，主世界毫发无损。

提示：零基础心法——别怕弄坏！Git 设计的初衷就是让你「大胆试错，随时后悔」。

专业映射：「存档」在 Git 里叫**提交（commit）**，每次提交记录的是整个文件夹在那一刻的完整样子——不是只记“改了哪几行”。「时光倒流」对应 Git 的回到历史版本功能，「平行宇宙」对应分支：分支只是一个小标签，指向某一次存档，想开一个就开一个，几乎不花任何成本。详见第 1 章与第 4 章。

0.1.2 GitHub 是「云端仓库」

代码放在自己电脑上，硬盘坏了或电脑丢了就全没了。GitHub 不是普通网盘，它不只是存文件，还记录谁改了什么、怎么合并、如何讨论。你可以把它理解成一个大家共同使用的「公共驿站」：你把自己的改动推上去，别人就能拉下来，所有人都能随时拿到最新版本。

- **备份**：电脑丢了、硬盘坏了，代码在 GitHub 上还在。
- **协作**：同事把代码传上来，你拉下来，合在一起，不用 U 盘传文件。
- **展示**：开源项目放在 GitHub，全世界都能看、能用、能提改进建议。

专业映射：Git 在设计上并没有「中心服务器」这个概念——每台电脑上的仓库地位是平等的，都能完整保存全部历史。GitHub 之所以显得像「云端仓库」，只是因为大家约定俗成，把它作为共同使用的集散地。详见第 5 章。

0.1.3 VS Code 是「驾驶舱」

汽车有仪表盘也有引擎盖——你开车不需要懂发动机原理，但仪表盘上的每个灯背后都有真实机械在运行。VS Code 就是 Git 的仪表盘：它把 Git 的复杂命令翻译成按钮、菜单、颜色竖线，你点按钮时，背后其实是一条条 Git 命令在替你执行。那些颜色竖线（绿色新增、蓝色修改、红色删除）就像仪表盘上的指示灯，随时告诉你引擎（Git）正在做什么。

- **不用背命令**：侧边栏点点鼠标，暂存、提交、推送、拉取、解决冲突全搞定。
- **所见即所得**：文件旁边的颜色竖线（绿 = 新增、蓝 = 修改、红 = 删除）实时告诉你改了啥。
- **AI 助手**：点「魔法棒」自动生成提交信息，遇到冲突 AI 也能帮忙分析。

专业映射：你在 VS Code 里点的每个按钮，背后都对应一条 Git 命令——点「提交」，就是替你执行了 `git commit`；点「推送」，就是替你执行了 `git push`。界面和命令只是同一件事的两种表达方式：学会看 VS Code 的输出，就能逐渐看懂命令行；反过来，懂一点命令行，也能更清楚 VS Code 在做什么。详见第 6 章。

0.2 Git 的设计哲学：四个关键词

类比帮你“想象”，但要真正理解 Git，需要四个关键词。每个关键词背后都对应 Git 设计中的一条根本取舍；本节只点出本质，细节留待第 1 章及后续章节展开。

0.2.1 关键词一：每个存档都有独一无二的编号

Git 给每次提交生成一个独一无二的编号，这个编号由文件内容本身算出来：内容一样编号就一样，哪怕只改一个标点，编号也会完全变。所以只要编号对得上，就能确信文件没有被悄悄改过。

0.2.2 关键词二：保存完整快照，不是只记差异

Git 每次提交保存的不是“改了哪几行”，而是整个项目在那一刻的完整样子。听起来很占空间，其实没变过的文件不会重复保存；需要看“改了什么”时，Git 再对比两个完整样子算给你看。

0.2.3 关键词三：历史是一张网，不是一条直线

每次提交都记着自己的“前一个提交”是谁。多人并行开发时，历史会分成好几条线，合并时再汇到一起——所以 Git 的历史是一张网，不是一条直线。正因为这样，多人同时开发互不干扰、合并也顺理成章。

0.2.4 关键词四：分支和标签只是小纸条

分支、标签在 Git 里只是一个个小标签，写着“我指向哪一次提交”。创建分支就是贴一个新标签，几乎不花任何成本——所以 Git 鼓励你多开分支、大胆尝试。

小结：编号保证数据可靠、完整保存保证历史清晰、网状历史保证并行开发、小标签保证操作轻快——这四个设计加起来，让 Git 成为版本控制的事实标准。版本控制系统的三代演进（RCS → CVS/SVN → Git）详见第 1 章表 1.1，BitKeeper 事件的来龙去脉见第 1 章正文。

0.3 本教程的学习路径图

教程共 10 章加附录，按认知规律四层递进：先用三个类比建立直观画面（第 0 章），再系统掌握 Git 的核心概念与基础命令（第 1–3 章），学成分支协作与远程同步，开启多人开发的真正威力（第 4–5 章），最后回到 VS Code 实战和完整工作流，让日常操作高效顺畅（第 6–7 章）。附录（第 8–9 章）跨阶段覆盖：避坑指南帮你躲开最常见的坑，命令速查卡供你随时翻阅。完整路径如图 2 所示——

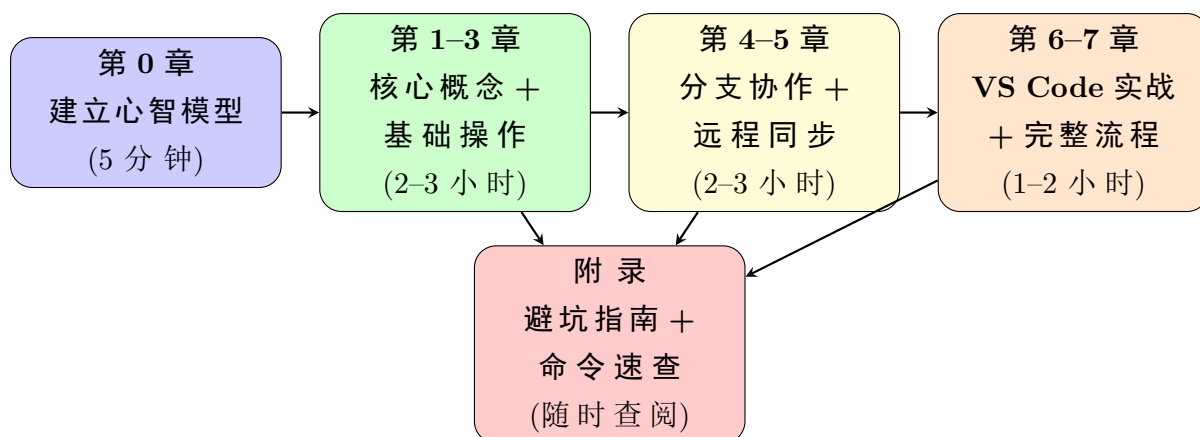


图 2: 本教程学习路径——从心智模型到实战工作流的章节安排

建议阅读策略:

- **极速上手 (30 分钟):** 只看第 0 章 → 第 1 章 1.1/1.2/1.5 → 第 2 章 2.1/2.2 → 第 3 章 3.1/3.4/3.5 → 第 7 章 7.2 (VS Code 操作流)
- **系统掌握 (1 周):** 按章节顺序完整阅读, 每章做完「动手实验」再进入下一章
- **遇到问题查阅:** 直接翻「附录: 避坑指南」或「命令速查表」

0.4 动手实验 0：验证环境就绪

在开始正式学习之前, 花两分钟确认工具箱就绪。下面四步全部通过, 说明 Git 和 VS Code 都已正确安装。

1. 打开终端 (Windows 用 PowerShell, Mac/Linux 用 Terminal)
2. 输入 `git --version`, 看到版本号即表示 Git 已安装
3. 输入 `code --version`, 看到版本号即表示 VS Code 已安装且已加入 PATH
4. 打开 VS Code, 按 `Ctrl+Shift+G` 打开源代码管理视图, 确认能看到界面

全部通过? 环境已就绪, 直接翻页进入第 1 章。

某一步失败? 别着急——翻到第 2 章跟着安装步骤走, 装完再回到这里继续; 整个过程不超过 10 分钟。

Chapter 1

版本控制与 Git 概述

本章学习目标：

1. 理解什么是版本控制，以及为什么需要版本控制
2. 掌握 Git 的核心特性和工作原理
3. 区分 Git 和 GitHub 的概念差异
4. 理解 Git 的三大工作区域及其关系
5. 了解 VS Code 中 Git 集成的方式

1.1 什么是版本控制

版本控制（Version Control）是一种记录文件内容变化历史、以便将来可以回溯到特定版本的系统。想象你在写论文：每隔一段时间，你会另存一个副本——”论文_v1.doc”、”论文_v2_final.doc”、”论文_v3_真的 final.doc”……版本控制系统就是自动帮你管理这些”副本”的工具，而且远比手动另存高效。

场景：一个没有版本控制的下午

假设你和小明、小红三个人合作写一个网站。你们约定好，每人负责一个页面：

- 你负责 `index.html`（首页）
- 小明负责 `about.html`（关于页面）
- 小红负责 `contact.html`（联系页面）

你们用微信传文件。下午 2 点，你把改好的 `index.html` 发到群里，小明下载后开始修改。下午 3 点，小红说她也需要改首页的导航栏，于是她也下载了 `index.html`。下午 4 点，小明改完了，把文件发回群里。下午 5 点，小红也改完了，也把文件发回群里。

现在问题来了：群里有两个版本的 `index.html`，它们都基于你下午 2 点的版本，但小明和小红各自做了不同的修改。你需要手动把两个人的修改”合并”到一起——打开两个文件，一行一行对比，复制粘贴……这个过程叫”手动合并”，耗时且极易出错。

更糟糕的是，如果你不小心复制错了，覆盖了某人的修改，这个修改就永远消失了，除非你记得具体内容，否则无法恢复。

版本控制系统就是为解决这些问题而生的。它自动记录每次修改的内容、时间和作者，让多人协作时不再”文件满天飞”，还能让你随时回到过去的任意版本。

在软件开发中，版本控制系统帮助开发者解决以下核心问题：

- **变更追踪：**记录每一次修改的内容、时间和作者，实现完整的修改历史。你可以精确地看到”谁在什么时候改了什么”。
- **协作开发：**多人同时修改同一项目时，自动合并（把平行宇宙的改动合回主世界）各自的改动，避免覆盖。不再需要”你改完了告诉我，我再改”的低效沟通。
- **安全回退：**当引入错误时，可以立即恢复到之前的稳定版本。就像游戏的”存档读档”功能。
- **并行开发：**通过分支（平行宇宙：在不影响主世界的情况下开发新功能）机制，在不影响主线的前提下同时进行多个功能的开发。
- **可追溯性：**每次修改都有记录，便于审计和问题定位。

版本控制的发展简史见表 1.1。

表 1.1: 版本控制的发展简史

时代	代表工具	年份	特点
本地版本控制	RCS	1982	单机使用，仅记录差异，无法协作
集中式版本控制	SVN, CVS	2000	中央服务器，需联网，服务器挂了就无法工作
分布式版本控制	Git, Mercurial	2005	每个克隆都是完整仓库，离线也能工作

1.2 Git 简介

Git 是由 Linus Torvalds（Linux 内核的创造者）于 2005 年开发的**分布式版本控制系统**。它的名字来源于英式英语中”愚蠢的人”的俚语——Linus 自嘲式的幽默。

在 Git 诞生之前，Linux 内核的维护者一直在寻找合适的版本控制工具。2002 年，Linux 内核项目开始使用 BitKeeper——一个当时先进的商业分布式版本控制系统。BitKeeper 为 Linux 社区免费提供使用许可，在大型项目的协作中发挥了重要作用。

然而到了 2005 年，BitKeeper 与 Linux 社区之间爆发了许可证冲突，开发者失去了免费使用权，内核的日常开发一度陷入困境。作为回应，Linus Torvalds 仅用了大约 4 天时间就写出了 Git 的原型，并迅速将其用于 Linux 内核的版本管理。Git 从一开始就被设计为 BitKeeper 的自托管替代方案，这也解释了它为何如此强调速度、分布式与数据完整性。集中式与分布式的对比如表 1.2 所示：

表 1.2: 集中式 vs 分布式版本控制

特性	集中式（如 SVN）	分布式（如 Git）
工作方式	所有修改提交到中央服务器	每个开发者拥有完整仓库副本
离线能力	无法提交、查看历史	可提交、查看历史、创建分支
分支成本	服务器复制文件，成本高	仅创建指针，成本极低
数据完整性	依赖中央服务器	SHA-1 哈希链保证
代表工具	SVN, CVS	Git, Mercurial

为什么 Git 能胜出？在 Git 之前，SVN 是主流。SVN 是集中式的——所有人的修改都要提交到同一台中央服务器。如果服务器宕机，所有人都无法工作。而 Git 是分布式的：每个人的电脑上都有一份**完整的**仓库副本（包括全部历史），即使断网也能提交、查看历史、创建分支。

如今，Git 已成为全球最广泛使用的版本控制系统，几乎所有开源项目和绝大多数商业软件团队都在使用它。截至 2025 年，Git 已成为全球开发者的标准工具。Stack Overflow 2024 年开发者调查显示，超过 93% 的专业开发者使用 Git 进行版本控制。GitHub 平台拥有超过 1 亿开发者用户，托管了数千万个开源项目。Linux 内核——世界上最大的 Git 仓库——包含超过 3000 万个文件对象和数百万次提交记录，充分证明了 Git 处理超大规模项目的能力。

1.2.1 Git 的核心特性

核心概念

1. **分布式架构**：每个开发者的机器上都有一份完整的代码仓库和历史记录，无需联网也能进行提交、查看历史、创建分支等操作。这意味着即使 GitHub 服务器宕机，你依然可以提交、查看历史、切换分支。
2. **快照式存储**：Git 不是记录文件差异（diff），而是对每次提交保存整个项目的快照。未修改的文件只保留指向旧快照的链接，因此效率极高。因此 checkout 任意历史版本是 $O(1)$ 操作，不受项目规模影响。
3. **强大的分支**：Git 的分支操作几乎是瞬时的（仅创建一个指针），创建、切换、合并分支的成本极低，鼓励频繁使用分支。创建一个分支仅占用 41 字节，这是 Git 鼓励频繁使用分支的根本原因。
4. **数据完整性**：每次提交都通过 SHA-1 哈希校验，任何篡改都会被检测到。SHA-1 哈希链保证任何历史篡改（哪怕是单字节）都会被立即检测。
5. **暂存区设计**：独特的暂存区（Staging Area）机制，允许精确控制每次提交包含哪些变更。这是 Git 区别于其他版本控制系统的重要特性。这让你可以将一个大改动拆分为多个语义清晰的提交，而非一个混乱的大提交。

理解了 Git 解决了什么问题以及它的设计哲学之后，让我们深入底层，看看 Git 是如何存储和寻址数据的。这一节的内容是理解 Git 命令背后原理的关键——你不必背诵，但理解后会让你在遇到问题时不再困惑。

1.3 Git 的内部数据模型

理解 Git 的底层数据模型，是掌握 Git 的关键。与其他版本控制系统不同，Git 本质上是一个 **** 内容寻址的文件系统 ****。理解这一点，你就能明白为什么 Git 如此强大。

1.3.1 Git 的四个核心对象

Git 仓库中的所有数据都存储为以下四种对象，如图 1.1 所示：

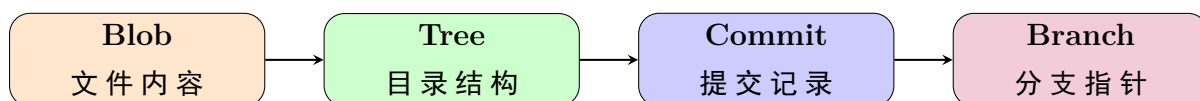


图 1.1: Git 的四个核心对象——Blob、Tree、Commit、Branch 的层次关系

- **Blob（二进制大对象）**：存储文件内容，不包括文件名。每个文件版本对应一个 Blob。
- **Tree（树）**：存储目录结构，记录文件名、权限和对应的 Blob 哈希。

- **Commit (提交)** (拍快照/结账: 生成一个不可改的版本号): 记录作者、时间、提交信息和父提交指针, 指向一个 Tree。
- **Branch (分支)**: 只是一个指向 Commit 的指针, 创建分支就是创建一个新指针。

Tag (标签): 指向特定 Commit 的固定引用, 通常用于标记版本号 (如 v1.0、v2.3)。Git 支持两种标签:

- **Annotated Tag (附注标签)**: 存储完整对象, 包含标签名、日期、注释和可选的 PGP 签名。创建命令: `git tag -a v1.0 -m "Release 1.0"`
- **Lightweight Tag (轻量标签)**: 只是指向 Commit 的指针 (等价于不会移动的分支), 不存储额外元数据。创建命令: `git tag v1.0`

1.3.2 HEAD 与引用

除了四种对象, Git 还使用引用 (refs) 来跟踪当前工作状态:

- **HEAD**: 指向当前检出的分支或 Commit。 `.git/HEAD` 文件内容通常是 `ref: refs/heads/main`, 表示当前在 `main` 分支上。
- **Detached HEAD**: 当 HEAD 直接指向某个 Commit 哈希 (而非分支名) 时, 称为“游离 HEAD”状态。此时修改不会关联到任何分支, 容易丢失。
- **分支引用**: `.git/refs/heads/` 目录下每个文件对应一个本地分支, 文件内容是该分支指向的 Commit 哈希。
- **标签引用**: `.git/refs/tags/` 目录下存放标签引用, Annotated Tag 指向 Tag 对象, Lightweight Tag 直接指向 Commit。

可以用以下命令查看 HEAD 和分支的关系:

<code>git log --oneline --all --graph</code>	图形化展示分支与 HEAD 关系
<code>git show-ref --heads</code>	查看所有分支引用
<code>git show-ref --tags</code>	查看所有标签引用

关键理解: Git 中的一切都是哈希。Blob、Tree、Commit 都有唯一的 SHA-1 哈希值 (40 位十六进制字符串)。Git 通过哈希值来寻址内容, 而不是文件名。这意味着: 如果两个文件内容相同, 无论文件名是否相同, Git 只存储一份 Blob。

验证内容寻址:

用底层命令验证: 同一内容无论文件名为何, Blob 哈希相同。

计算 "Hello Git" 内容的哈希

```
echo "Hello Git" | git hash-object --stdin
```

输出固定哈希值（例如：8ab686e...）

再次运行，输出相同哈希

```
echo "Hello Git" | git hash-object --stdin
```

这就是内容寻址的核心：Git 通过内容计算哈希，而非通过文件名定位。如果两个文件内容相同，无论文件名是否相同，Git 只存储一份 Blob。

1.3.3 一次提交的完整结构

一次提交的完整数据模型如图 1.2 所示：

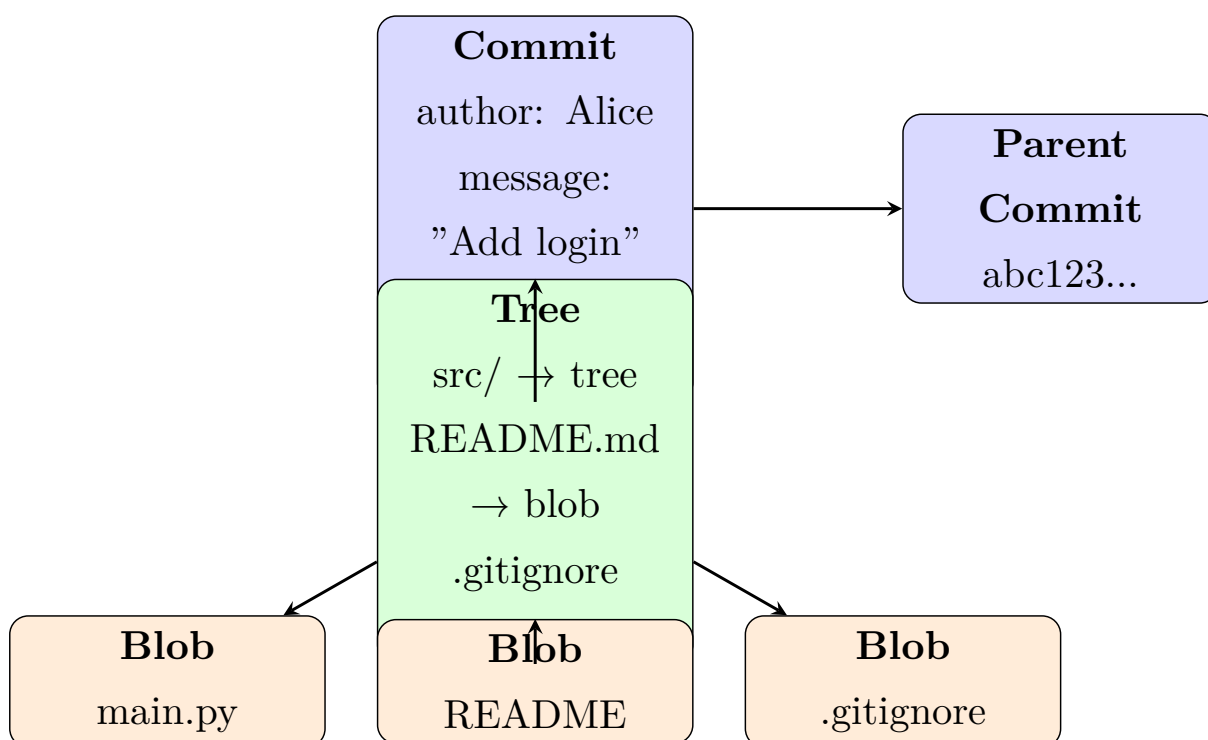


图 1.2: 一次提交的完整数据模型——Commit 指向 Tree，Tree 指向 Blob，并记录父提交

用底层命令查看对象：

Git 提供了底层命令来直接查看四种对象的内容：

<code>git cat-file -p HEAD</code>	查看 HEAD 指向的 Commit 对象
<code>git cat-file -p HEAD^{tree}</code>	查看该 Commit 对应的 Tree 对象
<code>git ls-tree HEAD</code>	列出 Tree 中的文件与 Blob 映射
<code>git cat-file -p <blob-hash></code>	查看 Blob 的内容

为什么 Git 高效？

- **快照式存储：**每次提交保存完整快照，未修改的文件复用旧 Blob（通过哈希判断）。
- **指针轻量：**分支只是一个 40 字节的指针，创建分支几乎不消耗空间。
- **本地完整：**每个克隆都包含完整历史，无需联网即可查看和操作。
- **自动垃圾回收：**Git 定期运行 `git gc`，将松散对象（loose objects）打包为 packfile 和索引文件，通过 delta 压缩进一步节省存储空间。

提示：Porcelain vs Plumbing——Git 命令分为两层**表层（Porcelain）**是你日常使用的命令（`git add`、`git commit`、`git log`），设计为人类友好；**底层（Plumbing）**是 Git 内部使用的命令（`git cat-file`、`git hash-object`、`git ls-tree`），输出格式稳定，适合脚本调用。理解数据模型时，底层命令是最好的”显微镜”。

日常工作中你几乎不需要直接使用 Plumbing 命令，但了解它们能帮你深入诊断问题。

1.4 GitHub 简介

GitHub 是全球最大的代码托管平台（2018 年被微软收购），截至 2025 年已有超过 1 亿开发者用户。它为 Git 仓库提供云端托管服务。Git 与 GitHub 的对比如表 1.3 所示：

表 1.3: Git vs GitHub 的区别

	Git	GitHub
本质	命令行工具（软件）	云端服务（网站）
类比	像”发邮件”这个动作	像”Gmail”这个服务
运行位置	你的电脑上	github.com 服务器上
是否需要联网	不需要	需要
替代品	Mercurial, SVN	GitLab, Bitbucket, Gitee

GitHub 提供的核心功能包括：

- **远程仓库托管（云端备份库）：**将本地 Git 仓库同步到云端，实现备份和多设备协作。

- **Pull Request** (PR, 合并申请: 请求把分支合并到主分支): 代码审查与合并请求机制, 是团队协作的核心流程。
- **Issue 管理**: 内建的任务追踪和 Bug 报告系统。
- **GitHub Actions**: CI/CD 自动化流水线, 支持自动测试和部署。
- **Pages**: 免费的静态网站托管服务。
- **Copilot**: AI 辅助编程工具, 集成在编辑器中。

1.5 Git 的三大工作区域

理解 Git 的关键在于理解它的三个工作区域。所有 Git 操作本质上都是在这三个区域之间移动数据。这是学习 Git 最重要的一张图, 请务必理解, 如图 1.3 所示:

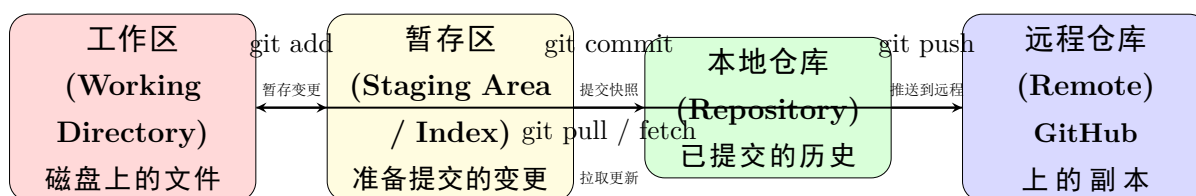


图 1.3: Git 三大工作区域与远程仓库——工作区、暂存区、本地仓库、远程仓库之间的数据流转

- **工作区 (Working Directory)**: 你在磁盘上看到的实际文件, 可以直接编辑。就是你在 VS Code 中打开的那些文件。
- **暂存区 (购物车: 选好要结账的商品; Staging Area / Index)**: 一个准备提交的变更列表。通过 `git add` 将文件放入暂存区, 只有暂存区的内容才会被包含在下一次提交中。暂存区是 Git 独有的设计——它让你可以精确选择“这次提交包含哪些修改”。
- **本地仓库 (Repository)**: Git 的 `.git` 目录, 存储了所有提交的完整历史。这是 Git 的“大脑”, 删除它等于删除所有版本历史。
- **远程仓库 (Remote)**: 托管在 GitHub 等服务器上的仓库副本。用于团队间的代码共享和备份。

提示: 类比理解——把 Git 想象成拍照。

- **工作区** = 你眼前的场景 (随时在变)
- **暂存区** = 取景框 (你选择要拍什么)

- 提交 = 按下快门（保存一个不可变的快照）

1.6 VS Code 如何与 Git 协作

VS Code 内置了强大的 Git 集成，将上述所有操作都图形化了。如图 1.4 所示，VS Code 源代码管理视图的全貌——你将在本教程中反复看到这个界面：

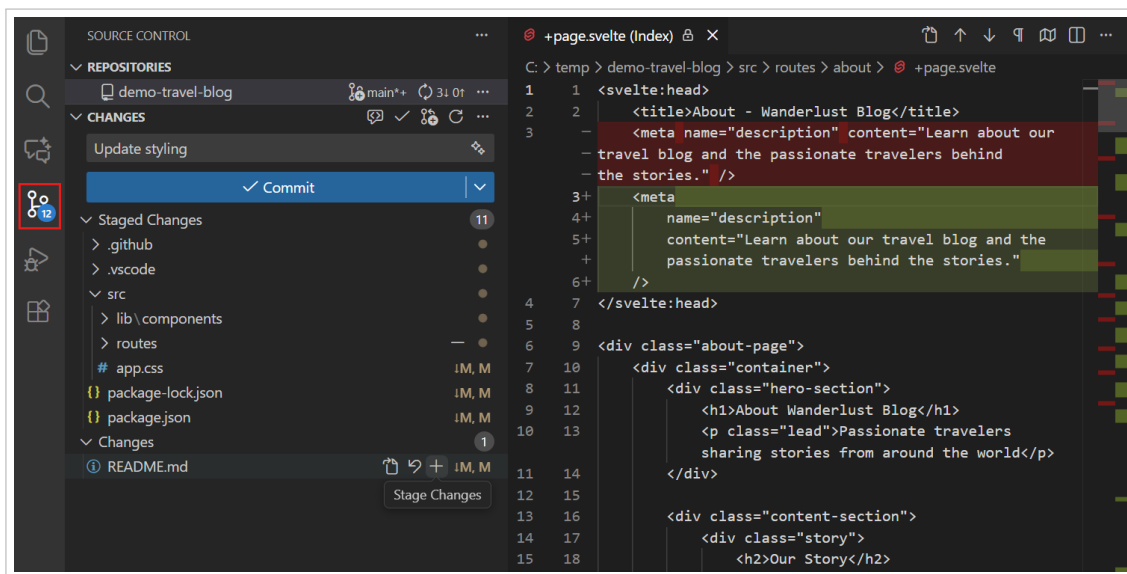


图 1.4: VS Code 源代码管理视图全景——左侧为变更文件列表，上方为提交信息输入框和提交按钮。这是你与 Git 交互的主要界面

在 VS Code 中，你可以通过点击按钮完成大部分 Git 操作，而不必记住命令行语法。但理解底层命令仍然重要——它们帮助你诊断问题、执行高级操作。

提示：本章小结

- 版本控制系统记录文件变更历史，支持协作、回退和并行开发
- Git 是分布式版本控制系统，每个克隆都包含完整历史
- Git 使用 Blob、Tree、Commit、Branch 四种对象存储数据
- GitHub 是云端托管平台，为 Git 仓库提供远程存储和协作功能
- Git 的三大工作区域：工作区 → 暂存区 → 本地仓库 → 远程仓库
- VS Code 内置了强大的 Git 集成，将命令行操作图形化

下一章预告：我们将学习如何安装 Git 和 VS Code，以及如何配置 GitHub 认证。

动手实验 1：初始化第一个仓库

花 5 分钟亲手创建本章反复提到的「仓库」。本实验全程只使用鼠标点击，不需要输入任何命令。

1. 在桌面空白处单击鼠标右键，选择「新建」→「文件夹」，输入名称 `my-first-repo` 并回车。
2. 打开 VS Code，点击菜单栏「文件」→「打开文件夹…」，选中 `my-first-repo` 文件夹，点击「选择文件夹」按钮。
3. 按 `Ctrl+Shift+G` 打开源代码管理视图，点击「初始化仓库」按钮。
4. 在左侧资源管理器点击「新建文件」图标，输入文件名 `hello.txt` 并回车；在编辑器中输入文字 `Hello Git!`，按 `Ctrl+S` 保存。
5. 回到源代码管理视图，观察 `hello.txt` 出现在「更改」列表中并带有 `U` 标记（Untracked，未跟踪），说明仓库已初始化成功。

提示：成功标志——打开 Windows 资源管理器进入 `my-first-repo` 文件夹，点击菜单栏「查看」→勾选「隐藏的项目」，可以看到 `.git` 文件夹。这个仓库会一直用于第 3、4 章的动手实验。

Chapter 2

Git 安装与配置

本章学习目标：

1. 在不同操作系统上安装 Git
2. 配置 Git 的用户信息和偏好设置
3. 安装和配置 VS Code
4. 配置 GitHub 认证（SSH 和 Token）

2.1 安装 Git

Git 版本选择建议

Git 是一个持续更新的工具，新版本通常修复了旧版本的 bug 并增加了功能。

- **Windows:** 建议从官网下载最新版（通常每几个月更新一次）
- **macOS:** 通过 Homebrew 安装可以方便地更新（`brew upgrade git`）
- **Linux:** 系统包管理器中的版本可能较旧，如需最新版可从官网下载源码编译

本书基于 Git 2.40+ 版本编写，但绝大多数操作在所有现代 Git 版本中都相同。

2.1.1 Windows 系统

1. 访问 Git 官网 <https://git-scm.com/download/win>，下载 Windows 安装包。
2. 运行安装程序，大部分选项保持默认即可。建议注意以下选项：

- **默认编辑器:** 选择”Use Visual Studio Code as Git’s default editor”。
- **PATH 环境:** 选择”Git from the command line and also from 3rd-party software”，这样可以在 PowerShell、CMD 等任何终端中使用 Git。
- **HTTPS 传输后端:** 选择”Use the OpenSSL library”。
- **行尾符号:** 选择”Checkout Windows-style, commit Unix-style line endings”。这能避免 Windows 和 Linux/macOS 之间的换行符冲突。

3. 安装完成后，打开 PowerShell，验证安装：

```
验证 Git 安装
git --version
输出示例: git version 2.45.0.windows.1
```

2.1.2 macOS 系统

方法一：通过 Xcode Command Line Tools（首次运行 git 时自动提示）
`xcode-select --install`

方法二：通过 Homebrew（推荐，版本更新）
`brew install git`

2.1.3 Linux 系统

Debian / Ubuntu
`sudo apt update && sudo apt install git`

Fedora / RHEL
`sudo dnf install git`

Arch Linux
`sudo pacman -S git`

2.2 基本配置

安装 Git 后，第一件事是配置你的身份信息和偏好设置。这些配置会附加在你每次提交上，相当于给每次修改”签名”。

为什么需要配置用户信息？

Git 是分布式版本控制系统，每次提交都会记录“谁”做了这次修改。如果没有配置用户信息，Git 会拒绝提交，并提示你设置 `user.name` 和 `user.email`。

这些信息会被永久记录在提交历史中，用于：

- 追溯代码的作者（`git log` 和 `git blame`）
- 与 GitHub/GitLab 等平台的账号关联
- 生成贡献统计图表

2.2.1 用户身份（必须配置）

设置全局用户名（显示在提交记录中）

```
git config --global user.name "你的名字"
```

设置全局邮箱（必须与 GitHub 账号邮箱一致）

```
git config --global user.email "your.email@example.com"
```

配置原理：这些设置被存储在你的用户目录下的 `.gitconfig` 文件中（Windows 在 C:

Users

你的用户名

`.gitconfig`，macOS/Linux 在 `~/.gitconfig`）。

每次执行 `git commit` 时，Git 会读取这个文件，将用户名和邮箱写入提交记录中。你可以随时用 `git config --global --list` 查看当前配置。

⚠ 注意

邮箱地址必须与你的 GitHub 账号关联的邮箱一致，否则 GitHub 无法将提交与你的账号关联。可以在 GitHub 的 Settings → Emails 中查看你的邮箱。

2.2.2 其他推荐配置

以下配置能显著提升日常使用体验：

设置默认分支名为 `main`（GitHub 的默认分支名）

```
git config --global init.defaultBranch main
```

设置默认编辑器为 VS Code（这样 `git commit` 会打开 VS Code 写提交信息）

```
git config --global core.editor "code --wait"
```



```
设置 pull 时使用 rebase（保持线性历史，推荐）
git config --global pull.rebase true

设置常用别名，简化命令输入
git config --global alias.st status           git st = git status
git config --global alias.co checkout         git co = git checkout
git config --global alias.br branch           git br = git branch
git config --global alias.ci commit           git ci = git commit
git config --global alias.lg "log --oneline --graph --all"
                                                    git lg = 图形化日志

查看当前所有配置
git config --list
```

2.2.3 配置层级

Git 配置有三个层级，优先级从高到低：（见表 2.1）

表 2.1: Git 配置的三个层级

层级	存储位置	作用范围
--local	仓库的.git/config	仅当前仓库生效
--global	~/.gitconfig	当前用户所有仓库生效
--system	/etc/gitconfig	系统所有用户生效

提示：日常使用中，绝大多数配置使用 --global 即可。如果某个项目需要特殊配置（比如用公司邮箱提交），在该仓库目录下不加 --global 即可覆盖全局设置。例如：git config user.email "work@company.com"（仅影响当前仓库）。

2.3 安装 VS Code

VS Code（Visual Studio Code）是微软开发的免费代码编辑器，内置了强大的 Git 集成。

为什么选择 VS Code？

VS Code 之所以成为最受欢迎的代码编辑器之一，除了免费开源外，还有以下优势：

- **内置 Git 集成：**无需额外安装插件，开箱即用

- **直观的 Diff 视图：**清晰展示文件修改前后的对比
- **图形化冲突解决：**用颜色标记冲突区域，一键选择保留哪边
- **丰富的扩展生态：**GitLens、Git Graph 等扩展进一步增强了 Git 体验
- **跨平台：**Windows、macOS、Linux 都有完全一致的界面和操作

1. 访问 <https://code.visualstudio.com/> 下载并安装。
2. 安装时勾选”将 Code 注册为受支持的文件类型的编辑器”和”添加到 PATH”。
3. 安装完成后，可以通过命令行 `code` 在当前目录打开 VS Code。

2.4 配置 GitHub 认证

VS Code 与 GitHub 集成需要认证。推荐使用以下方式之一：

2.4.1 方式一：GitHub CLI（推荐）

安装 GitHub CLI (<https://cli.github.com/>)

Windows: `winget install GitHub.cli`

macOS: `brew install gh`

登录 GitHub

`gh auth login`

按提示选择: GitHub.com -> HTTPS -> Login with browser

2.4.2 方式二：Personal Access Token

1. 登录 GitHub → Settings → Developer settings → Personal access tokens → Tokens (classic)。
2. 点击”Generate new token (classic)”，勾选 `repo` 权限。
3. 复制生成的 token，在 Git 提示输入密码时粘贴。
4. 建议使用 `git credential-manager` 保存凭据，避免重复输入。

提示：VS Code 首次连接 GitHub 时，会自动弹出登录提示，引导你通过浏览器完成 OAuth 认证。认证成功后，VS Code 会自动管理 token，无需手动配置。

2.4.3 方式三：SSH 密钥（最推荐）

SSH 密钥认证是最安全、最方便的 GitHub 认证方式。配置一次后，无需每次都输入密码。

配置 SSH 密钥：

1. 打开终端，生成 SSH 密钥（使用你的 GitHub 邮箱）：

生成 SSH 密钥（使用你的 GitHub 邮箱）

```
ssh-keygen -t ed25519 -C "your.email@example.com"
```

按 `Enter` 接受默认保存位置

输入并确认一个密码（可选，但推荐）

2. 将 SSH 密钥添加到 `ssh-agent`：

启动 `ssh-agent`

```
eval "$(ssh-agent -s)"
```

添加私钥到 `ssh-agent`

```
ssh-add ~/.ssh/id_ed25519
```

3. 复制公钥内容：

复制公钥到剪贴板（macOS）

```
pbcopy < ~/.ssh/id_ed25519.pub
```

或手动复制（Windows / Linux）

```
cat ~/.ssh/id_ed25519.pub
```

然后手动复制输出内容

4. 在 GitHub 网页上添加公钥：

- 登录 GitHub → Settings → SSH and GPG keys
- 点击“New SSH key”
- 粘贴公钥内容，给密钥起个描述性标题（如“My Laptop”）
- 点击“Add SSH key”

5. 测试连接：

```
ssh -T git@github.com
```

看到 `"Hi username! You've successfully authenticated..."` 表示成功

HTTPS vs SSH 对比：

- **HTTPS:** 使用用户名 + 密码（或 Token）认证。适合临时使用，但频繁推送时需要输入凭据。
- **SSH:** 使用密钥对认证。配置一次后免密操作，适合长期开发。

推荐: 个人开发使用 SSH，团队协作可配合使用 HTTPS。

动手实验 2：配置用户身份和 SSH 密钥

花 5 分钟完成身份配置并添加 SSH 密钥。本实验全程只使用鼠标和浏览器，不需要输入命令。配置后你的每次提交都会正确显示作者身份，并且可以用 SSH 免密访问 GitHub。

1. 打开 VS Code，按 **Ctrl+O**（打开文件），在输入框粘贴路径 `C:\Users\你的用户名\.gitconfig`，点击「打开」；若提示文件不存在，点击弹窗中的「创建文件」按钮。
2. 在文件中输入以下内容（把名字和邮箱换成你自己的，邮箱须与 GitHub 账号一致），按 **Ctrl+S** 保存：

```
[user]
  name = 你的名字
  email = your.email@example.com
```

3. 打开浏览器登录 <https://github.com>，点击右上角头像 → 「Settings」 → 左侧菜单「SSH and GPG keys」。
4. 点击绿色按钮「New SSH key」，在 Title 输入框输入 `My Laptop`。
5. 在 Key 输入框粘贴第 2 章生成的公钥内容（`id_ed25519.pub` 文件的内容），点击「Add SSH key」完成添加。

提示: 成功标志——SSH 密钥页面出现名为 `My Laptop` 的密钥记录；同时 VS Code 中提交信息会显示你在 `.gitconfig` 中填写的名字和邮箱。

Chapter 3

Git 基础操作

本章学习目标：

1. 掌握仓库的初始化和克隆操作
2. 理解文件的四种状态及其转换关系
3. 熟练使用 status、add、commit 等核心命令
4. 掌握 diff 视图的三种对比模式
5. 学会撤销和回退操作
6. 配置.gitignore 文件

本章介绍 Git 最核心的日常操作。无论你使用命令行还是 VS Code 图形界面，底层执行的都是这些命令。理解它们，你就能在任何场景下灵活应对。

3.1 初始化仓库

每个 Git 项目都始于一个**仓库**（Repository）。

什么是仓库？

你可以把”仓库”理解为一个”时间胶囊”——它不仅存储了你当前的文件，还存储了这些文件的每一次修改记录。当你对于一个文件进行修改、保存、提交后，Git 会把这次修改记录下来，就像一个时间戳，标记着”在这个时刻，文件是这样的”。

初始化仓库，就是告诉 Git：”从现在开始，请帮我记录这个文件夹里所有文件的变化历史。”

有两种方式创建仓库：

3.1.1 方式一：在已有项目中初始化

如果你已经有一个项目文件夹，想将它纳入 Git 管理：

进入项目目录

```
cd my-project
```

初始化 Git 仓库（创建 .git 目录）

```
git init
```

此时目录中会出现一个隐藏的 .git 文件夹

这就是 Git 存储所有历史数据的地方

在 VS Code 中，操作更加直观——打开项目文件夹后，源代码管理视图会自动检测到这不是一个 Git 仓库，并显示“初始化仓库”按钮，如图 3.1 所示：

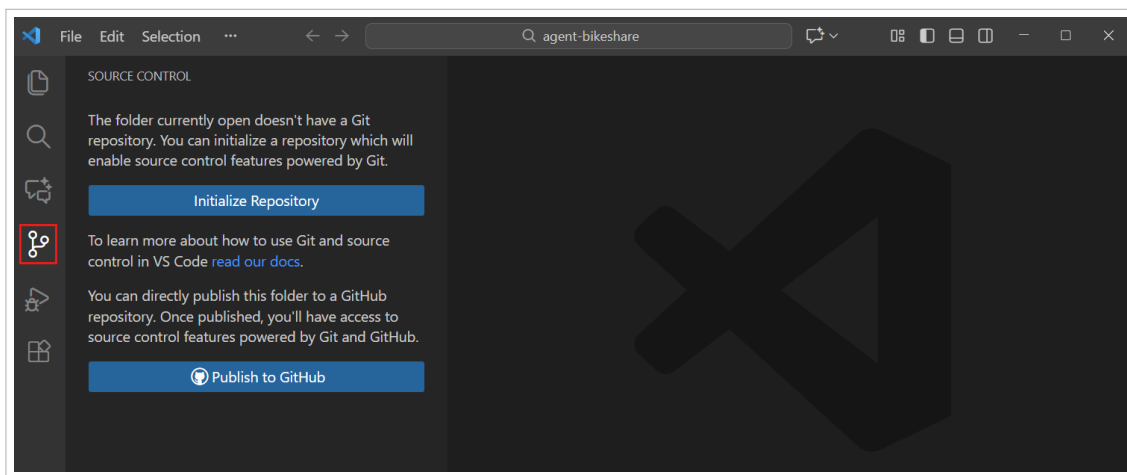


图 3.1: VS Code 中的“初始化仓库”按钮——打开一个非 Git 项目时，源代码管理视图会提示你初始化

点击后，VS Code 会在项目根目录执行 `git init`，创建 .git 目录。此时你会看到所有项目文件出现在“更改”列表中，状态为 U（Untracked，未跟踪）。

3.1.2 方式二：克隆远程仓库

如果项目已经存在于 GitHub 等远程平台，使用 `git clone`（克隆：把云端仓库完整下载到本地）将完整仓库下载到本地：

克隆远程仓库到本地

```
git clone https://github.com/username/repo-name.git
```

克隆到指定目录

```
git clone https://github.com/username/repo-name.git my-folder
```

克隆时指定分支

```
git clone -b develop https://github.com/username/repo-name.git
```

在 VS Code 中，可以通过命令面板执行克隆操作，如图 3.2 所示：

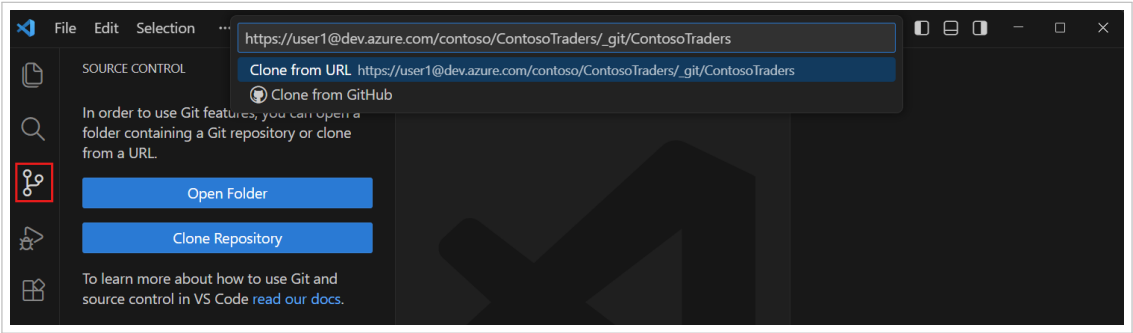


图 3.2: VS Code 的 Git: Clone 命令——按 Ctrl+Shift+P 输入”clone”，粘贴仓库 URL 即可克隆

git clone 实际上做了三件事：

- 1. 从远程下载完整的仓库历史到本地（包括所有分支和提交）
- 2. 创建名为 origin 的远程引用指向原仓库
- 3. 自动检出（checkout）默认分支（通常是 main）的文件到工作区

3.2 文件状态与跟踪

Git 中的文件有四种状态，理解这些状态是掌握 Git 的基础：如表 3.1 所示。

表 3.1: Git 中文件的四种状态

状态	标记	含义
Untracked（未跟踪）	??	新文件，Git 尚未纳入管理，不会出现在提交中
Unmodified（未修改）	无标记	已跟踪，且自上次提交后未修改
Modified（已修改）	M	已跟踪，但内容已被修改，需要决定是否提交
Staged（已暂存）	A / M	修改已放入暂存区，将在下次提交时保存

文件状态的生命周期如图 3.3 所示：

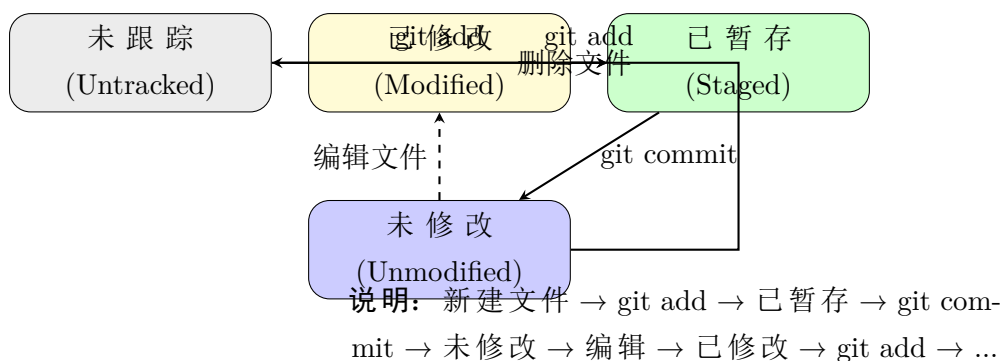


图 3.3: 文件状态生命周期——未跟踪、已修改、已暂存、未修改之间的转换关系

3.3 日常工作流：status → add → commit

这是你最频繁使用的 Git 操作循环——修改文件 → 暂存 → 提交，每天会重复几十次。

3.3.1 第一步：查看状态（git status）

查看仓库状态（最常用！）

```
git status
```

简洁输出（推荐日常使用）

```
git status -s
```

示例输出：

```
M modified-file.txt    ← 已修改，未暂存（红色）
A new-file.txt         ← 新文件，已暂存（绿色）
?? untracked.txt       ← 未跟踪的文件（红色）
```

在 VS Code 中，git status 的信息直接体现在源代码管理视图里，如图 3.4 所示：

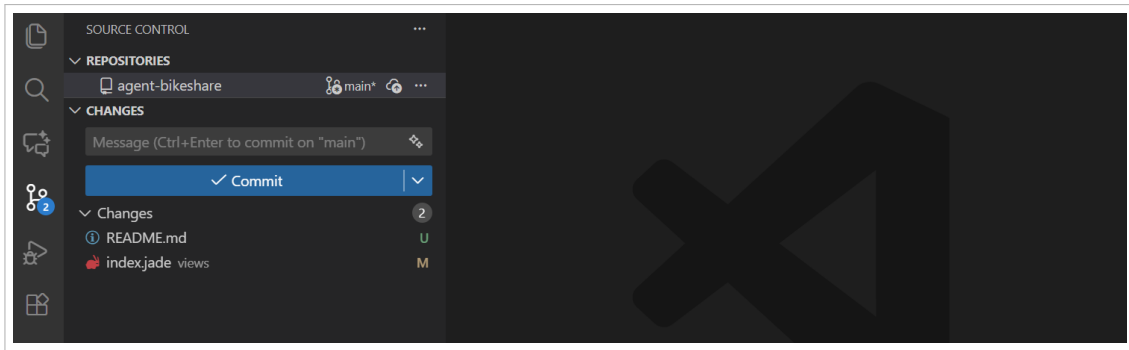


图 3.4: 修改后的文件列表——源代码管理视图的”更改”区域显示所有已修改但未暂存的文件，每个文件旁有状态标记

图 3.4 中，你可以看到：

- 每个修改过的文件都列在”更改”区域下
- 文件名右侧的字母表示状态（M = Modified, U = Untracked）
- 点击文件名可以打开差异对比视图，查看具体改了什么

提示： `git status` 是你最应该频繁执行的命令。它告诉你当前处于什么状态，哪些文件需要处理。养成”做任何操作前先 status”的习惯。在 VS Code 中，状态信息始终可见——源代码管理视图就是实时的 `git status`。

3.3.2 第二步：暂存文件（git add）

暂存是 Git 独有的设计——它让你在提交之前**精确选择**要包含哪些修改。

暂存单个文件

```
git add filename.txt
```

暂存多个文件

```
git add file1.txt file2.txt file3.txt
```

暂存当前目录下所有变更（包括新文件和修改）

```
git add .
```

暂存指定目录

```
git add src/
```

查看暂存区中具体有哪些变更

```
git diff --cached
```

在 VS Code 中，暂存操作就是点击文件旁边的“+”按钮（单个暂存），或点击“更改”标题右侧的“+”按钮（全部暂存），如图 3.5 所示：

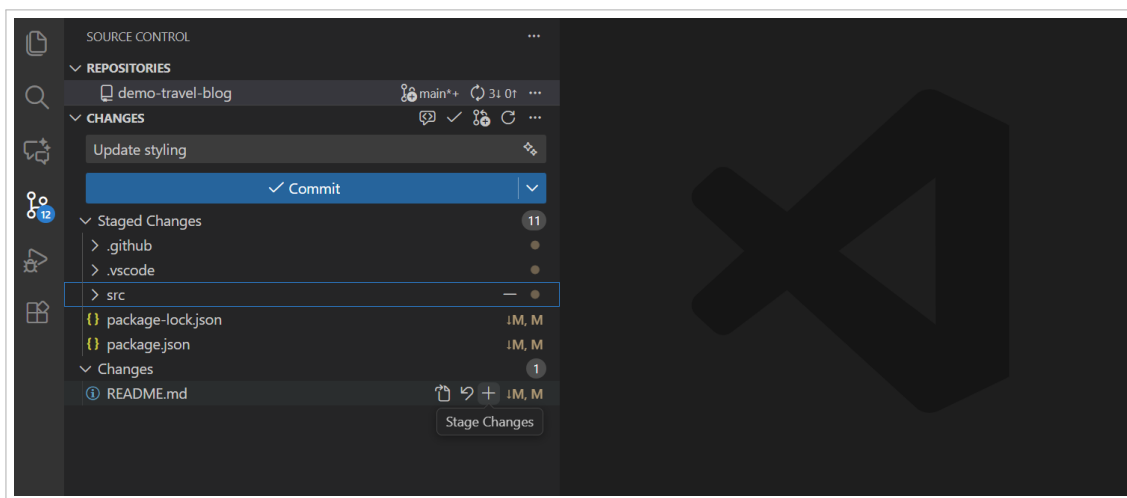


图 3.5: 暂存操作——点击文件右侧的“+”按钮暂存单个文件，或点击“更改”标题旁的“+”按钮全部暂存

⚠ 注意

`git add .`（或 VS Code 中的全部暂存按钮）会暂存**所有**变更，包括你可能不想提交的文件（如临时文件、密钥文件等）。请务必先用 `git status` 确认，并通过 `.gitignore` 文件排除不需要的文件。

3.3.3 第三步：提交变更（git commit）

提交暂存区的变更（会打开编辑器让你写提交信息）

```
git commit
```

直接在命令行写提交信息

```
git commit -m "添加用户登录功能"
```

提交所有已跟踪文件的修改（跳过 `git add` 步骤）

```
git commit -a -m "修复样式问题"
```

在 VS Code 中，提交操作分两步：输入提交信息 → 点击提交按钮，如图 3.6 所示。

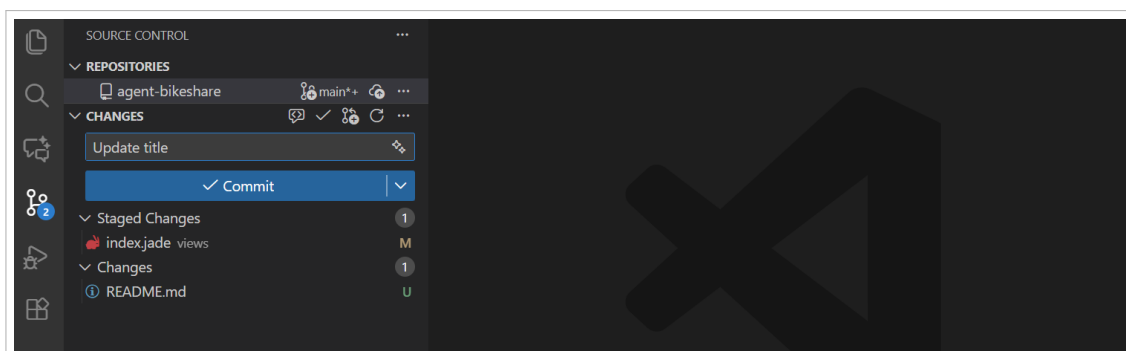


图 3.6: 提交操作——在顶部输入框填写提交信息，然后点击”提交”按钮（或按 Ctrl+Enter）完成提交

提交后，”暂存的更改”区域清空，变更进入本地历史记录。

3.3.4 编写好的提交信息

提交信息（commit message）是 Git 历史中最重要的文档。好的提交信息应该：

- **第一行：**简明扼要地描述变更（不超过 50 个字符），使用祈使语气。
- **空一行后（可选）：**详细描述为什么做这个改动，而不仅仅是做了什么。

好的提交信息示例

```
git commit -m "修复购物车总价计算精度问题"
git commit -m "添加用户头像上传功能"
git commit -m "重构数据库连接池配置"
```

差的提交信息示例

```
git commit -m "fix"           太模糊——修了什么？
git commit -m "update"        更新了什么？
git commit -m "asdfasdf"      无意义
git commit -m "修复了bug，改了三个文件，还加了一个功能"  太杂，应拆分为多个提交
```

提示：VS Code 支持 AI 辅助生成提交信息——点击提交信息输入框旁的”魔法棒”图标，VS Code 会根据你的代码变更自动生成描述。你可以在此基础上修改，也可以直接采纳。

3.4 查看修改内容：Diff 视图

在提交之前，你应该先检查自己到底改了什么。VS Code 提供了强大的可视化差异对比功能。

3.4.1 打开 Diff 视图

在源代码管理视图中，点击文件名即可打开差异对比视图，如图 3.7 所示：

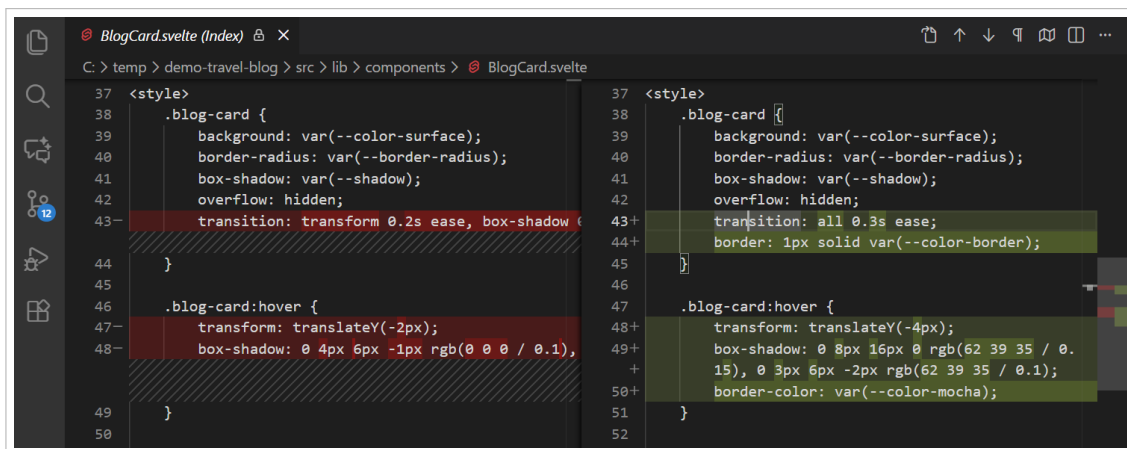


图 3.7: 差异对比编辑器——左侧为旧版本（HEAD），右侧为当前修改。绿色背景表示新增内容，红色背景表示删除内容

3.4.2 并排视图 vs 内联视图

VS Code 支持两种差异对比模式，点击 Diff 编辑器右上角的按钮可以切换：

- 并排视图（Side-by-Side）：左右分栏显示两个版本，适合大幅修改。
- 内联视图（Inline）：所有变更在同一文件中显示，适合小改动。

两种模式的对比如图 3.8 与图 3.9 所示——并排视图可以一目了然地看到左右差异，而内联视图则更紧凑：

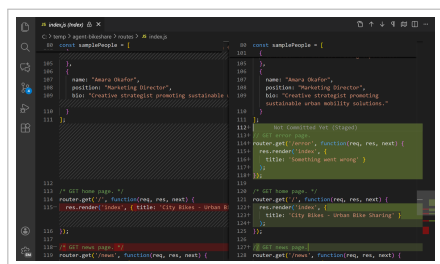


图 3.8: 并排视图——左右分栏，适合大幅修改

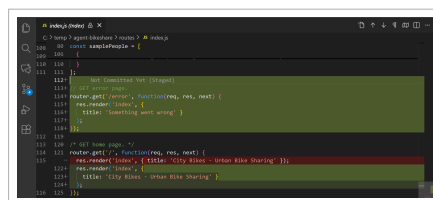


图 3.9: 内联视图——变更在同一文件中显示，适合小改动

3.4.3 编辑器中的实时变更指示

在代码编辑器中，VS Code 会在行号右侧的 gutter 区域实时显示变更指示，如图 3.10 所示：

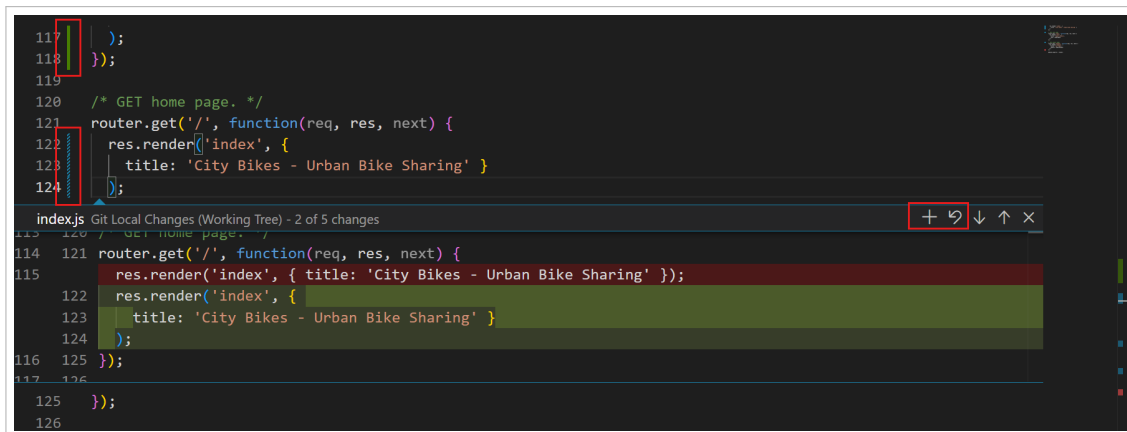


图 3.10: 编辑器 Gutter 变更指示器——绿色竖线表示新增行，蓝色竖线表示修改行，红色三角表示删除行。点击可查看详细信息

- 绿色竖线：新增的行
- 蓝色竖线：修改的行
- 红色三角：删除的行（行号旁的小三角）

点击这些指示器，会弹出一个小浮窗，显示该行的详细变更内容，并提供“撤销此更改”的快捷操作。

3.4.4 命令行 diff 详解

除了 VS Code 的可视化 diff，命令行的 `git diff` 也非常强大。掌握以下常用用法：

查看工作区与暂存区的差异（未暂存的修改）

```
git diff
```

查看暂存区与最后一次提交的差异（已暂存的修改）

```
git diff --cached
```

```
git diff --staged    同上
```

查看工作区与最后一次提交的所有差异（暂存+未暂存）

```
git diff HEAD
```

查看两次提交之间的差异

```
git diff abc123..def456
```

查看某个文件的差异

```
git diff filename.txt
```

查看某次提交的完整变更

```
git diff abc123^..abc123
```

diff 的输出格式解读:

---	a/filename.txt	旧版本（减号开头）
+++	b/filename.txt	新版本（加号开头）
@@	-1,5 +1,7 @@	变更位置：从第1行开始，旧版本5行，新版本7行
	unchanged line	无符号 = 未变更
-	removed line	减号 = 删除的内容
+	added line	加号 = 新增的内容
	unchanged line	上下文行，帮助定位

3.5 查看历史：git log

基本日志（显示完整信息：作者、日期、提交信息）

```
git log
```

简洁模式：每行一个提交（推荐）

```
git log --oneline
```

输出示例：

a1b2c3d 添加用户登录功能

e4f5g6h 修复购物车计算bug

i7j8k9l 初始化项目

图形化显示分支历史

```
git log --oneline --graph --all
```

查看最近 N 条提交

```
git log -5
```

```
git log --oneline -10
```

查看某个文件的修改历史

```
git log -p filename.txt
```

搜索提交信息

```
git log --grep="关键词"
```

查看某人的提交

```
git log --author="名字"
```

按时间范围

```
git log --since="2024-01-01" --until="2024-06-30"
```

3.6 撤销操作

Git 提供了多种撤销手段，适用于不同场景。这是初学者最容易困惑的部分，但理解了“三个工作区域”的概念后，撤销操作就很好理解了。

3.6.1 撤销工作区的修改（未 add）

场景：你改了一个文件，但改错了，想恢复到上次提交的状态。

丢弃工作区中对某个文件的修改（恢复到上次提交的状态）

```
git checkout -- filename.txt
```

或者使用更直观的新语法（Git 2.23+）

```
git restore filename.txt
```

⚠ 注意

这个操作是不可逆的！被撤销的修改无法恢复。请确认你真的不需要这些修改。

3.6.2 取消暂存（已 add，未 commit）

场景：你暂存了文件，但发现暂存了不该暂存的文件，想把它从暂存区移回去。

将文件从暂存区移回工作区（保留修改，只是不再暂存）

```
git reset HEAD filename.txt
```

或者使用新语法

```
git restore --staged filename.txt
```

提示：在 VS Code 中，取消暂存就是点击“暂存的更改”区域中文件旁边的“-”按钮。

3.6.3 修改上一次提交

场景：你刚提交完，发现提交信息写错了，或者忘了暂存某个文件。

修改上一次提交的信息

```
git commit --amend -m "新的提交信息"
```

将新的修改追加到上一次提交中（不增加新提交）

```
git add forgotten-file.txt
```

```
git commit --amend --no-edit
```

⚠ 注意

`git commit --amend` 会改变提交的哈希值。如果这个提交已经推送到远程，需要强制推送（`git push --force-with-lease`），谨慎使用。

3.6.4 回退到之前的版本

回退提交，保留修改在工作区（安全，推荐）

```
git reset --soft HEAD~1
```

 回退1个提交，修改回到暂存区

回退提交，保留修改在工作区（未暂存）

```
git reset --mixed HEAD~1
```

 默认行为

回退提交，丢弃所有修改（危险！不可恢复！）

```
git reset --hard HEAD~1
```

创建一个新的提交来"撤销"指定提交（安全，推荐用于已推送的提交）

```
git revert HEAD
```

```
git revert <commit-hash>
```

reset vs revert 的选择原则：

- **reset**：修改历史。适用于**未推送**的本地提交。
- **revert**：添加新提交来撤销。适用于**已推送**的提交，不会破坏他人的历史。

简单记忆：已经 push 的用 revert，没 push 的用 reset。

3.7 .gitignore 文件

.gitignore 文件告诉 Git 哪些文件或目录不应该被跟踪。在项目根目录创建名为 .gitignore 的文件：

```
.gitignore 示例

依赖目录（这些文件由包管理器生成，不需要提交）
node_modules/
vendor/
__pycache__/_

编译输出
*.exe
*.dll
*.o
*.pyc
build/
dist/

编辑器和 IDE 配置
.vscode/           如果要共享设置则不忽略
.idea/
*.swp
*.swo

操作系统文件
.DS_Store          macOS
Thumbs.db          Windows

环境变量和密钥（永远不要提交！）
.env
.env.local
*.pem
*.key

日志文件
*.log
logs/
```

 注意

永远不要提交密钥文件（API key、密码、私钥等）到 Git 仓库！如果不小心提交了，即使后续删除，它仍然存在于 Git 历史中。需要使用 `git filter-repo` 工具彻底清除，并立即更换泄露的密钥。

提示：.gitignore 的常见模板——GitHub 提供了大量现成的.gitignore 模板。创建新仓库时可以直接选择模板，或访问 <https://github.com/github/gitignore> 下载。例如 Python 项目、Node.js 项目、Java 项目都有对应的模板。

提示：本章小结

- 通过 `git init` 初始化仓库，`git clone` 克隆远程仓库
- 文件有四种状态：未跟踪、未修改、已修改、已暂存
- 核心工作流：修改 → `git add` → `git commit`
- 用 `git diff` 查看修改内容，用 `git log` 查看历史
- 撤销操作：`git restore`（工作区）、`git restore --staged`（暂存区）、`git commit --amend`（提交）
- .gitignore 用于排除不需要跟踪的文件

下一章预告：我们将学习 Git 最强大的特性——分支与合并。

动手实验 3：修改-暂存-提交完整流程

继续使用实验 1 的仓库，花 5 分钟走一遍本章的核心循环「修改 → 暂存 → 提交」。全程只使用鼠标点击。

1. 在 VS Code 中打开 `hello.txt`，把内容修改为 `Hello Git, version 2!`，按 `Ctrl+S` 保存。
2. 按 `Ctrl+Shift+G` 打开源代码管理视图，观察 `hello.txt` 出现在「更改」列表中并带有 M 标记（Modified，已修改）。
3. 点击 `hello.txt` 文件名，打开 Diff 对比视图，确认绿色行为新增内容，核对无误后关闭标签页。
4. 点击 `hello.txt` 右侧的「+」按钮，将其暂存；文件随即移动到「暂存的更改」区域。
5. 在顶部提交信息输入框中输入更新问候语，点击「提交」按钮（或按 `Ctrl+Enter`）；提交后「暂存的更改」区域清空，说明变更已保存进本地历史。

提示：成功标志： 点击源代码管理视图标题栏的「提交历史图」图标，能看到一条提交记录更新问候语。

Chapter 4

分支与合并

本章学习目标：

1. 理解分支的本质和工作原理
2. 掌握创建、切换、合并、删除分支的操作
3. 学会解决合并冲突
4. 理解 rebase 的作用和使用场景
5. 了解常见的分支策略（Feature Branch、GitFlow）

4.1 分支的概念

分支（Branch）是 Git 最强大的特性之一。一个分支本质上只是一个指向某个提交的可移动指针。创建分支不会复制任何文件，只是创建了一个新指针——因此几乎不消耗时间和空间。

HEAD 指针：HEAD（当前所在的平行宇宙入口）是一个特殊指针，指向你当前所在的分支。当你切换分支时，HEAD 移动到新分支，工作区的文件也随之更新为该分支最后一次提交的状态。

```
A --- B --- C (main, HEAD)
      \
        D --- E (feature)
```

上图中，HEAD 指向 main 分支，main 指向提交 C，feature 分支指向提交 E。当你在 feature 分支上提交时，feature 指针会向前移动，但 main 保持不动。

为什么要用分支？ 假设你正在开发一个功能，需要三天才能完成。如果直接在 main 上开发，这三天的半成品代码会一直“污染”主线。使用分支，你可以在独立的平行世界中开发，完成后一次性合并回去。

分支的本质类比： 想象你在玩一个 RPG 游戏，主线剧情是 main 分支。当你想尝试一个支线任务时，你保存了当前进度（创建分支），然后在支线任务中探索。如果支线任务失败（功能开发失败），你可以随时回到主线进度（切换回 main），不会丢失任何主线进度。如果支线任务成功（功能完成），你可以将支线任务的奖励合并到主线中（合并分支）。

这就是分支的核心价值：**隔离风险，并行探索。**

4.2 分支操作

4.2.1 创建、切换、删除分支

创建新分支（不切换）

```
git branch feature-login
```

切换到已有分支

```
git switch feature-login
```

或旧语法：git checkout feature-login

创建并立即切换（最常用！）

```
git switch -c feature-login
```

或旧语法：git checkout -b feature-login

查看所有本地分支（* 标记当前分支）

```
git branch
```

查看所有分支（包括远程分支）

```
git branch -a
```

删除已合并的分支（安全删除）

```
git branch -d feature-login
```

强制删除未合并的分支（谨慎！会丢失未合并的修改）

```
git branch -D feature-login
```

在 VS Code 中，分支操作非常直观——点击底部状态栏的分支名称，会弹出分支选择列表，你可以在其中切换或创建新分支，如图 4.1 所示：

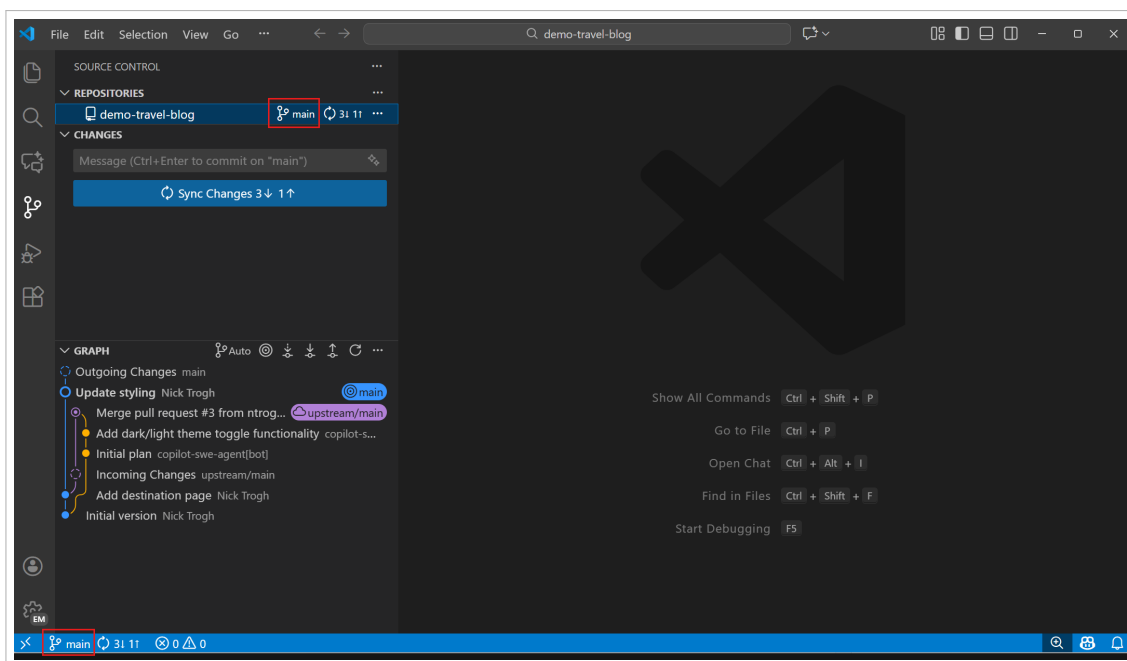


图 4.1: 底部状态栏的分支指示器——显示当前所在分支名称（如“main”），点击可切换或创建分支

创建新分支时，在弹出的输入框中输入分支名称即可（如 `feature/login`），VS Code 会自动从当前分支创建并切换，如图 4.2 所示：

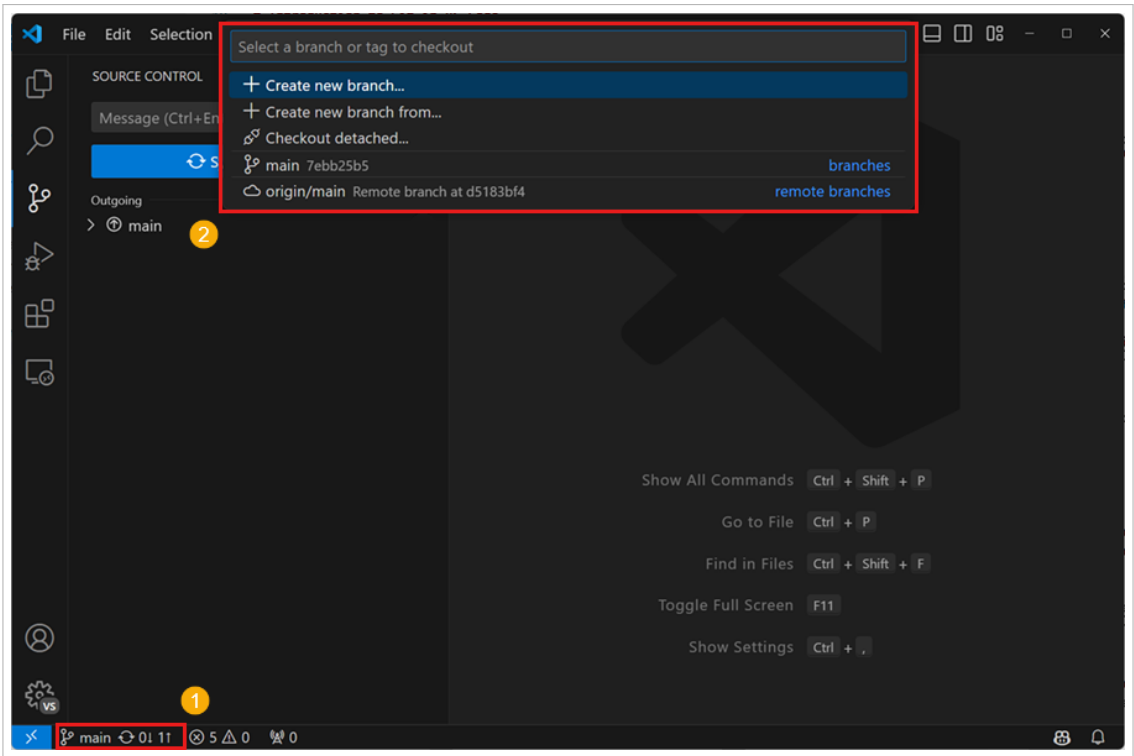


图 4.2: 创建新分支——输入分支名称后，VS Code 会自动从当前分支创建并切换

4.2.2 分支命名规范

常见的分支命名规范如表 4.1 所示：

表 4.1: 常见分支命名规范

类型	命名示例	用途
功能分支	feature/user-login	开发新功能
修复分支	fix/null-pointer	修复 Bug
热修复	hotfix/security-patch	紧急修复生产环境
发布分支	release/v1.2.0	准备发布版本

4.3 合并分支

当功能开发完成后，需要将功能分支合并回主分支。

合并的本质：合并（merge）是将两个分支的历史整合到一起的过程。想象两条河流（分支）在下游汇合（合并点），Git 会尝试自动将两边的修改整合到一起。如果两边的修改没有冲突（比如修改了不同的文件或同一文件的不同部分），Git 会自动完成合并；如果两边修改了同一处内容，Git 就会标记冲突，需要人工介入决定保留哪边的修改。

4.3.1 合并的两种类型

先切换到目标分支（要合并到的分支）

```
git switch main
```

合并功能分支

```
git merge feature-login
```

1. 快进合并 (Fast-forward): 如果目标分支没有新的提交，Git 只是简单地将指针前移。没有创建新的提交，历史是线性的。

合并前：A --- B --- C (main)
 \
 D --- E (feature)

合并后：A --- B --- C --- D --- E (main, feature)

2. 三方合并 (3-way merge): 如果两个分支都有新提交，Git 找到共同祖先，创建一个合并提交（有两个父节点）。

合并前：A --- B --- C --- F (main)
 \
 D --- E (feature)

合并后：A --- B --- C --- F --- G (main)
 \ /
 D --- E (feature)

G 是合并提交，有两个父节点 F 和 E

4.4 解决合并冲突

当两个分支修改了同一文件的同一部分时，Git 无法自动合并，会产生冲突（两个平行宇宙改了同一处，Git 不知道听谁的）。这在新手看来很可怕，但其实解决冲突非常简单。

4.4.1 冲突标记

冲突发生时，Git 会在文件中插入冲突标记（如图 4.3 所示）：

```
<<<<<<< HEAD
```

这是当前分支（main）的内容

=====

这是要合并的分支（feature）的内容

>>>>>> feature

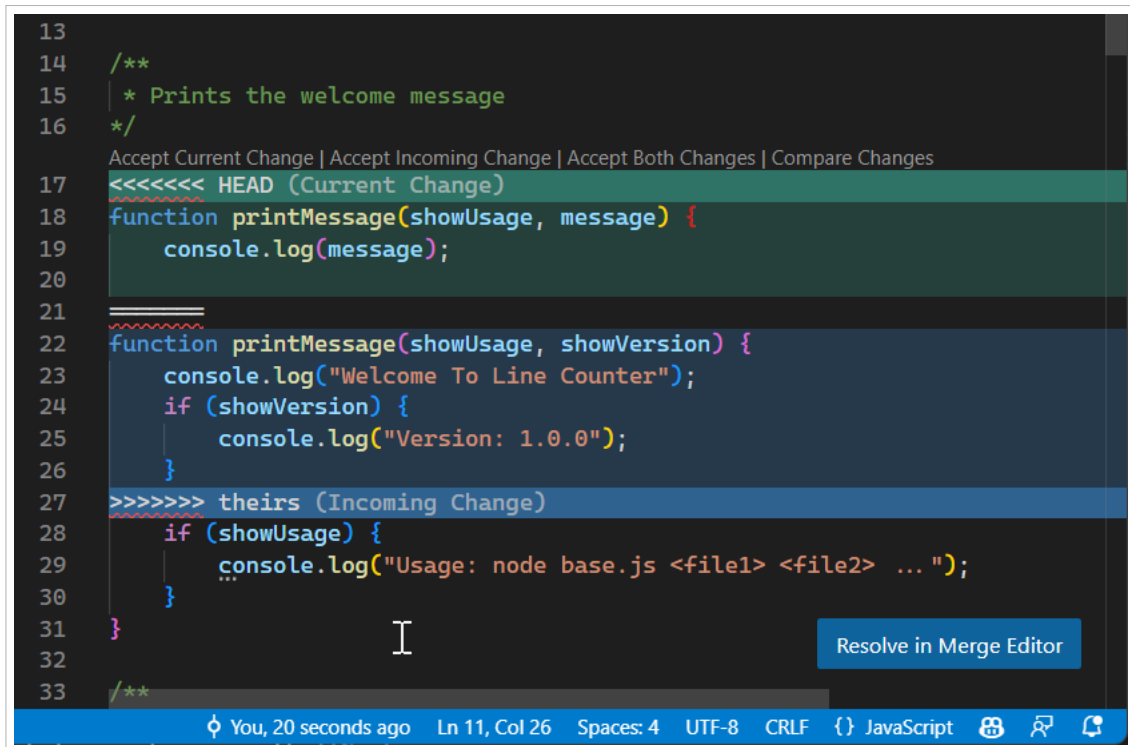


图 4.3: VS Code 中的合并冲突标记——文件中用特殊颜色标记冲突区域，并提供快捷操作按钮

4.4.2 VS Code 的冲突解决体验

VS Code 在遇到合并冲突时，会在每个冲突区域上方提供快捷操作按钮：

- **Accept Current Change:** 保留当前分支的内容
- **Accept Incoming Change:** 保留要合并分支的内容
- **Accept Both Changes:** 保留两者的内容（按顺序拼接）
- **Compare Changes:** 打开并排对比视图，仔细查看差异

VS Code 还提供了专门的三方合并编辑器，如图 4.4 所示：

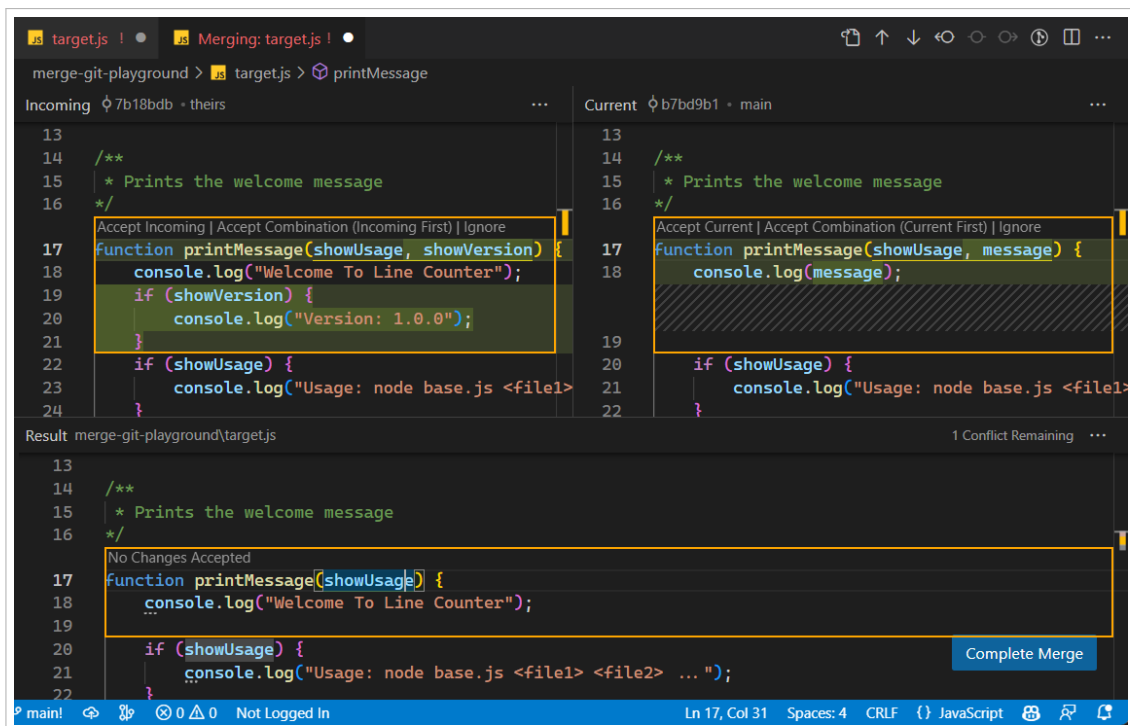


图 4.4: VS Code 的三方合并编辑器——左侧为当前分支（Current），右侧为传入分支（Incoming），底部为合并结果

4.4.3 解决冲突的完整步骤

1. 合并后，VS Code 自动打开有冲突的文件。
2. 在源代码管理视图中，冲突文件标记为！（感叹号）。
3. 逐个处理冲突区域，使用上述按钮或手动编辑。
4. 删除所有冲突标记（<<<<<<<、=====、>>>>>>>）。
5. 保存文件。
6. 在源代码管理视图中，点击冲突文件旁的“+”暂存（标记为已解决）。
7. 输入提交信息，点击“提交”完成合并。

查看哪些文件有冲突

```
git status
```

解决冲突后

```
git add conflicted-file.txt
```

```
git commit 完成合并（Git 会自动生成合并提交信息）
```

如果想放弃合并

```
git merge --abort
```

提示：搜索冲突：按 Ctrl + Shift + P，输入”Merge Conflict”，可以看到所有冲突相关的命令，包括”Accept All Current”等批量操作。

4.4.4 远程协作中的冲突（PR 冲突）

前面讲的是本地合并冲突。但在团队协作中，还有一种更常见的场景：你创建了一个 Pull Request（PR），继续在该分支上工作并再次 push 后，PR 页面突然提示”This branch has conflicts that must be resolved”。这不是你的 push 失败了，而是远程仓库的历史发生了分叉。

PR 冲突的产生原因

想象两条铁路从同一个车站（共同祖先）出发，分别向不同方向延伸。一条是 main 分支，另一条是你的 feature 分支。当你创建 PR 后，main 上可能又有其他人提交了新的代码，而你的 feature 分支也继续提交了新的修改。如果两边的修改恰好落在同一文件的同一区域，Git 就无法自动合并，PR 页面就会提示冲突。

用时间线表示，冲突的产生过程如下：

```
main:      A --- B --- C --- D          <- PR 后 main 新增 C、D
feature: A --- B --- E --- F          <- 分支上继续提交 E、F
           |
           | 双方在 B 后分叉，改到同一处 -> PR 冲突
```

i 说明

PR 冲突的本质和本地冲突一样：两个分支修改了同一文件的同一部分，Git 不知道听谁的。区别在于，本地冲突发生在你执行 `git merge` 时，而 PR 冲突发生在远程仓库尝试合并时。

PR 冲突与本地冲突的区别

- **发生位置不同：**本地冲突在你的电脑上；PR 冲突在远程仓库（GitHub/Gitee 服务器端）。
- **触发时机不同：**本地冲突是执行 `git merge` 时触发；PR 冲突是 PR 页面尝试合并时检测到的。

- **解决方式不同：**本地冲突直接在本地解决后提交；PR 冲突需要先同步远程最新代码，解决后再 push 更新 PR。

 **核心概念** PR 冲突不是你的 push 失败了，而是 PR 双方的历史分叉所致。只要将远程 main 的最新内容同步到你的分支，解决冲突后再 push，PR 就会恢复正常。

解决 PR 冲突的方法

解决 PR 冲突有两种方式：命令行和 VS Code 图形界面。

方式一：命令行

1. 先获取远程最新状态

```
git fetch origin
```

2. 方式一：用 rebase 把本地提交变基到最新 main 上（推荐，历史整洁）

```
git rebase origin/main
```

解决冲突后：git add <文件> 再 git rebase --continue

最后强制推送更新 PR：git push --force-with-lease

2. 方式二：用 merge 合并远程 main 到本地分支

```
git merge origin/main
```

解决冲突后：git add <文件> 再 git commit

最后推送更新 PR：git push

注意

rebase 会重写提交历史。如果分支已推送到远程且被他人使用，不要 rebase，改用 merge；否则会导致他人的本地分支与远程不一致。

 **提示** --force-with-lease 比 --force 更安全：它只在远程没有新提交时才强制覆盖，避免覆盖他人的工作。

方式二：VS Code 图形界面

在 VS Code 中解决 PR 冲突的步骤如下：

1. 点击源代码管理视图顶部的“同步”按钮（或执行 git fetch origin），获取远程最新代码。
2. 执行 rebase 或 merge 操作后，VS Code 会显示冲突标记和快捷操作按钮（如图 4.3 所示）。

3. 使用"Accept Current Change"、"Accept Incoming Change" 等按钮，或手动编辑冲突区域。
4. 删除所有冲突标记 (<<<<<<<、=====、>>>>>>>)，保存文件。
5. 在源代码管理视图中，点击冲突文件旁的"+" 暂存（标记为已解决）。
6. 输入提交信息，点击"提交" 完成合并。
7. 最后 push 更新 PR。

i 说明

PR 冲突不是分支坏了，而是双方改了同一处代码，解决后推送即可。核心思路是：先同步远程最新代码，解决冲突，再推送更新 PR。

预防措施

1. **经常同步 main**：定期执行 `git fetch origin`，将 main 的最新内容同步进自己的分支（通过 rebase 或 merge）。
2. **小步快跑**：提交要小、PR 要小，冲突面就小。一个 PR 只解决一个问题，避免大段代码长时间不合并。
3. **合并前再确认**：PR 合并前再次同步 main，确认无冲突后再点"Merge"。
4. **提前沟通**：大改动前先和团队沟通改动范围，避免多人同时改同一文件。

4.5 Rebase: 整理历史

Rebase（变基，把提交历史整理成一条直线）是另一种整合分支的方式。与 merge 不同，rebase 会"重写"提交历史，将当前分支的提交"移动"到目标分支的最新提交之后。

4.5.1 Rebase 与 Merge 的区别

两者对比如图 4.5 所示——

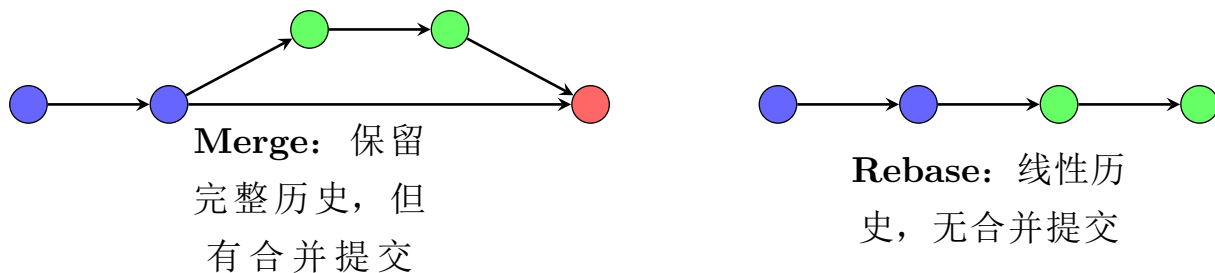


图 4.5: Merge 与 Rebase 的对比——Merge 保留完整历史产生合并节点，Rebase 生成线性历史

4.5.2 Rebase 的使用场景

将 feature 分支变基到 main 分支的最新提交上

```
git switch feature
git rebase main
```

交互式 rebase（整理提交历史）

```
git rebase -i HEAD~3
```

可以选择：合并（squash）、修改（reword）、删除（drop）提交

如果 rebase 过程中出现冲突

1. 解决冲突
2. `git add .`
3. `git rebase --continue`

放弃 rebase

```
git rebase --abort
```

⚠ 注意

Rebase 的黄金法则：永远不要对已经推送到远程的公共分支执行 rebase！Rebase 会改写提交历史，如果其他人已经基于旧历史做了工作，rebase 后会导致混乱。

Merge vs Rebase 的选择：

- **Merge:** 保留完整历史，适合团队协作的公共分支（如 main）
- **Rebase:** 整理历史，适合个人开发的功能分支（在推送前）
- **推荐做法：**本地开发时用 rebase 整理历史，合并到 main 时用 merge

4.6 Cherry-pick: 挑选提交

Cherry-pick 允许你将某个分支上的特定提交，复制到当前分支。

将 abc123 这个提交应用到当前分支

```
git cherry-pick abc123
```

Cherry-pick 多个提交

```
git cherry-pick abc123 def456
```

Cherry-pick 时如果出现冲突，解决后：

```
git add .
```

```
git cherry-pick --continue
```

Cherry-pick 的使用场景：

- 修复了 main 上的 bug，需要将修复同步到 release 分支
- 从功能分支中挑选某个提交到另一个功能分支
- 误删了某个提交，需要重新应用

4.7 分支策略简介

4.7.1 Feature Branch 工作流（推荐）

这是最常用、最推荐的团队协作工作流：

1. main 分支始终保持可发布状态。
2. 每个新功能从 main 创建独立的 feature/* 分支。
3. 功能完成后，通过 Pull Request 合并回 main。
4. 合并后删除功能分支。

```
main:    A --- B ----- G --- H （稳定）
          \           /
feature:  C --- D --- E --- F
```

4.7.2 GitFlow 工作流

适合有固定发布周期的项目，使用多种长期分支：

- **main**: 生产环境代码，每次合并代表一次发布。
- **develop**: 开发主线，集成所有功能。
- **feature/***: 从 **develop** 创建的功能分支。
- **release/***: 发布准备分支。
- **hotfix/***: 从 **main** 创建的紧急修复分支。

对于个人项目或小团队，Feature Branch 工作流已经足够。GitFlow 更适合有严格发布流程的大型项目。

提示：本章小结

- 分支是指向提交的可移动指针，创建和切换分支几乎不消耗资源
- 合并分支有 Fast-forward 和 3-way merge 两种方式
- 合并冲突时，需要手动解决冲突标记，然后暂存并提交
- Rebase 可以整理历史，但只应在未推送的本地分支上使用
- Cherry-pick 可以挑选特定提交应用到当前分支
- Feature Branch 是最常用的分支策略，GitFlow 适合大型项目

下一章预告：我们将学习如何将本地仓库与远程仓库同步，以及如何在 GitHub 上进行协作。

动手实验 4：创建分支、合并、解决冲突

花 5 分钟体验一次「分支上开发 → 合并 → 解决冲突」的完整过程。全程只使用鼠标点击，不需要输入任何命令。

1. 点击 VS Code 底部状态栏左侧的分支名称 **main**，在弹出的菜单中点击「Create new branch…」，输入 **feature/hello** 并回车（此时已自动切换到新分支）。
2. 修改 **hello.txt**，把内容改为 **feature** 分支的内容，保存；点击文件旁「+」暂存，输入提交信息 **feature 修改**，点击「提交」。

3. 再次点击底部状态栏分支名，切回 `main` 分支；把 `hello.txt` 的同一行改为 `main` 分支的内容，保存、暂存，输入提交信息 `main` 修改并提交。
4. 点击源代码管理视图顶部的「…」菜单 → 「分支」→ 「合并分支…」，选择 `feature/hello` 并确认；由于两个分支都改了同一行，VS Code 弹出冲突提示，文件标记变为!
5. 打开冲突文件，在冲突区域上方点击「Accept Both Changes」按钮，按 `Ctrl+S` 保存，点击文件旁「+」暂存，输入提交信息解决冲突，点击「提交」完成合并。

提示：成功标志：底部状态栏分支名为 `main`，源代码管理视图无待处理更改。若想放弃合并，点击「…」菜单 → 「分支」→ 「中止合并」即可。

Chapter 5

远程仓库与 GitHub

本章学习目标：

1. 理解远程仓库的概念和作用
2. 掌握 push、pull、fetch 的区别和使用
3. 学会在 GitHub 上创建和管理仓库
4. 理解 Pull Request 的工作流程
5. 了解 Fork 和贡献开源项目的方法

5.1 远程仓库的概念

远程仓库（Remote）是托管在服务器上的 Git 仓库副本。最常用的是 `origin`，通常指向 GitHub 上的仓库。你可以有多个远程仓库（比如同时托管在 GitHub 和 GitLab）。

本地仓库 vs 远程仓库：

想象你在家写小说（本地仓库），同时你还将手稿复印了一份寄给出版社（远程仓库）。出版社的编辑可以阅读你的小说，但不能直接修改你家里的原稿——除非你把他们的修改“拉取”回来。

在 Git 中：

- **本地仓库：**存储在你电脑上的完整仓库副本，包含所有历史和分支
- **远程仓库：**存储在服务器（如 GitHub）上的仓库副本，用于团队协作和备份
- **origin：**默认的远程仓库名称，就像 `main` 是默认分支名一样

查看远程仓库

```
git remote -v
```

输出示例：

```
origin https://github.com/username/repo.git (fetch)
```

```
origin https://github.com/username/repo.git (push)
```

添加远程仓库

```
git remote add origin https://github.com/username/repo.git
```

修改远程仓库地址

```
git remote set-url origin https://github.com/username/new-repo.git
```

删除远程仓库

```
git remote remove origin
```

5.2 推送与拉取

5.2.1 git push: 上传本地提交到远程

推送当前分支到远程

```
git push origin main
```

首次推送新分支时，设置上游追踪（之后只需 git push）

```
git push -u origin feature-login
```

之后只需

```
git push
```

推送所有分支（谨慎使用）

```
git push --all
```

5.2.2 git fetch: 下载远程更新（不合并）

从远程下载所有更新（不影响工作区，安全操作）

```
git fetch origin
```

查看远程分支的状态

```
git branch -r
```

5.2.3 git pull: 下载并合并远程更新

```
git pull = git fetch + git merge
```

```
git pull origin main
```

推荐使用 rebase 模式（保持线性历史）

```
git pull --rebase origin main
```

设置全局默认使用 rebase

```
git config --global pull.rebase true
```

fetch vs pull:

- **fetch:** 只下载，不修改你的工作区。安全操作，可以先查看再决定。
- **pull:** 下载并立即合并。更方便，但可能产生意外的合并提交。

推荐做法：日常使用 `git pull --rebase`，需要谨慎时使用 `fetch` + 手动 `merge`。

在 VS Code 中，推送（把本地提交上传到云端）和拉取（从云端下载最新并合并）操作可以通过底部状态栏的同步按钮一键完成，如图 5.1 与图 5.2 所示：

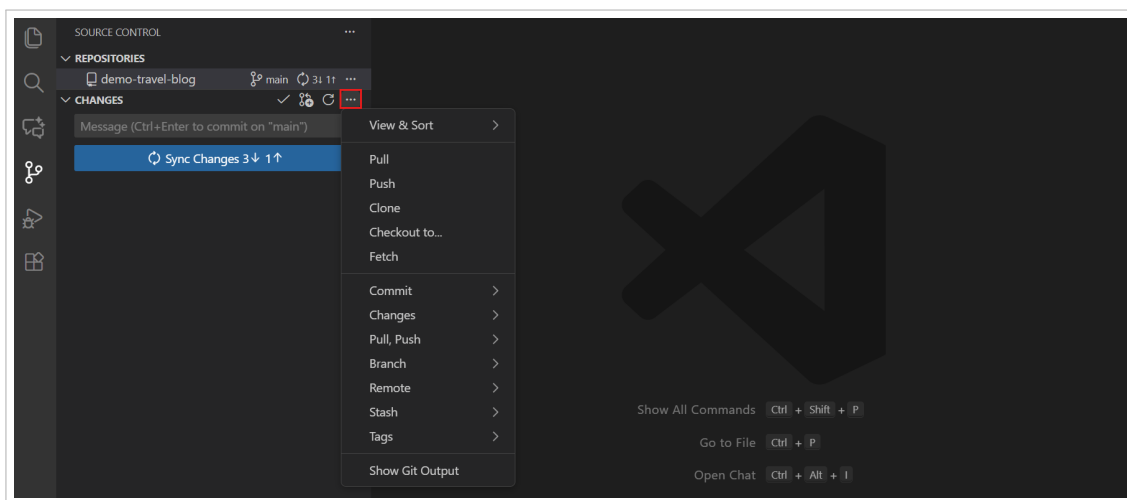


图 5.1: Pull 和 Push 操作——在源代码管理视图的“...”菜单中可以单独执行拉取或推送

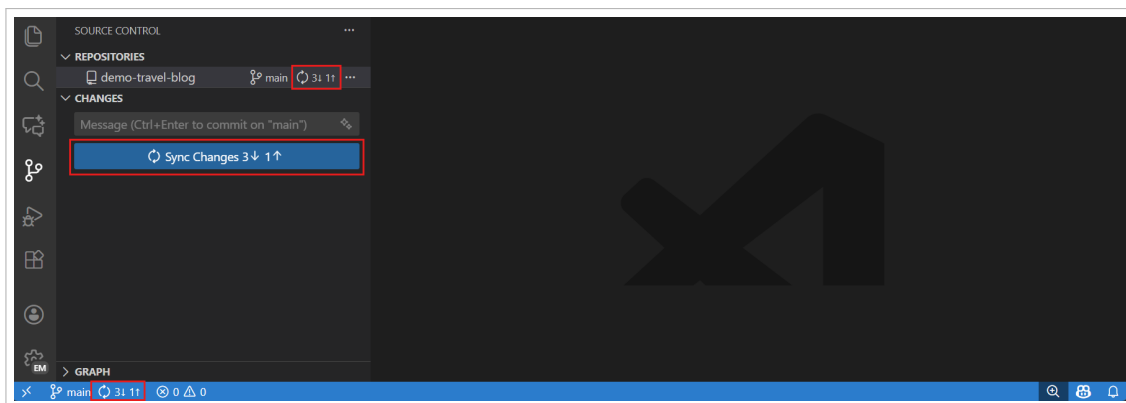


图 5.2: 同步按钮——源代码管理视图顶部的同步操作按钮，点击可执行一键同步（pull + push）

5.3 在 GitHub 上创建仓库

5.3.1 方式一：先在 GitHub 创建，再克隆到本地

1. 登录 GitHub，点击右上角“+” → “New repository”。
2. 填写仓库名称、描述，选择 Public 或 Private。
3. 勾选“Add a README file”（可选，但推荐）。
4. 点击“Create repository”。
5. 在本地克隆：`git clone https://github.com/username/repo.git`

5.3.2 方式二：先在本地创建，再推送到 GitHub

1. 在 GitHub 上创建空仓库（不勾选 README、.gitignore 等）。
2. 在本地项目中初始化 Git 并推送：

```
cd my-project
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/username/repo.git
git push -u origin main
```

5.3.3 方式三：VS Code 一键发布

VS Code 可以一步完成”初始化 + 提交 + 创建远程仓库 + 推送”，如图 5.3 所示：

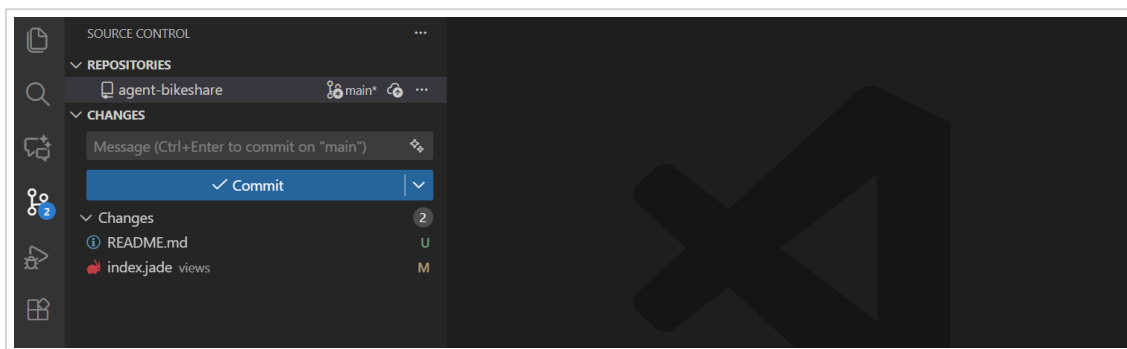


图 5.3: VS Code 一键发布——对于未初始化的项目，VS Code 提供”初始化仓库”按钮，之后可以直接”发布到 GitHub”

1. 打开源代码管理视图。
2. 点击”**Initialize Repository**”（初始化仓库）。
3. 暂存并提交你的文件。
4. 点击”**Publish Branch**”（发布分支）。
5. VS Code 会询问你要创建公开仓库还是私有仓库。
6. 选择后，VS Code 会自动在 GitHub 上创建仓库并推送代码。
7. 点击通知中的”Open on GitHub” 查看你的仓库。

5.4 Pull Request (PR)

Pull Request 是 GitHub 的核心协作功能。它不是 Git 命令，而是 GitHub 的网页功能。PR 的本质是：”我做了些修改，请你审查并合并到主分支。”

为什么需要 Pull Request？

想象你正在为一个开源项目贡献代码。如果没有 PR 流程，你只能直接推送代码到主分支，这可能导致：

- 未经审查的代码直接进入生产环境
- 团队成员不知道你做了什么修改

- 无法追踪某个功能是由谁、在什么时候、为什么引入的

PR 流程解决了这些问题：

1. **代码审查**：在合并前，其他开发者可以审查你的代码，提出改进建议
2. **自动化测试**：PR 可以触发自动化测试，确保不会破坏现有功能
3. **讨论记录**：PR 页面保留了所有讨论和修改历史，方便追溯
4. **知识共享**：团队成员可以通过审查他人的 PR 学习新的编码技巧

简单记忆：PR = 代码的”体检报告”，确保代码健康后才合并。

5.4.1 PR 的标准流程

1. 从 main 创建功能分支：`git switch -c feature/xxx`
2. 在功能分支上开发并提交。
3. 推送功能分支到 GitHub：`git push -u origin feature/xxx`
4. 在 GitHub 网页上点击”Compare & pull request”。
5. 填写 PR 标题和描述，指定审查者 (Reviewer)。
6. 审查者查看代码、提出修改意见 (Review)。
7. 根据反馈修改代码，推送更新 (PR 自动更新)。
8. 审查通过后，点击”Merge pull request” 合并。
9. 删除已合并的功能分支。

提示：好的 PR 描述应包含：

- 这个 PR 解决了什么问题？
- 做了什么改动？
- 如何测试这些改动？
- 相关的 Issue 编号（如 Closes #123）

5.4.2 PR 工作流程图

Pull Request 的完整工作流程如图 5.4 所示：

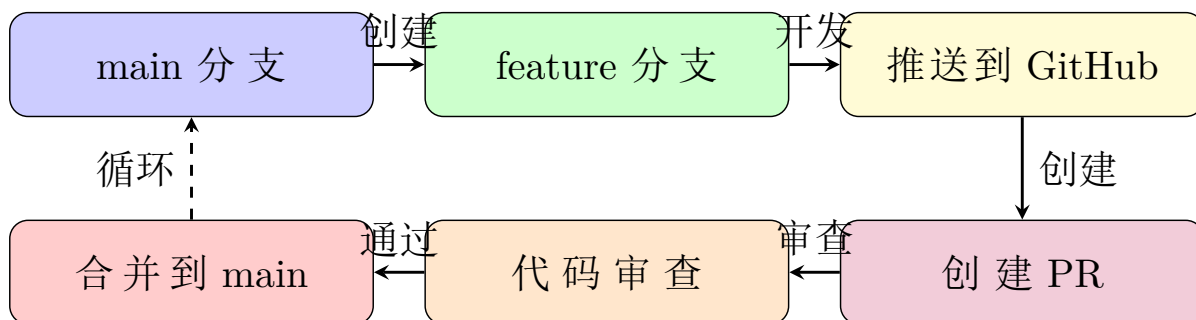


图 5.4: Pull Request 完整工作流程——从创建分支到代码审查再到合并的协作循环

5.5 Fork 与贡献开源项目

如果你想为别人的开源项目贡献代码：

1. 在 GitHub 上点击“Fork”（复制别人的仓库到自己账号下），将项目复制到你自己的账号下。
2. 克隆你的 Fork: `git clone https://github.com/your-username/repo.git`
3. 添加上游远程: `git remote add upstream https://github.com/original/repo.git`
4. 创建功能分支，开发并提交。
5. 推送到你的 Fork: `git push origin feature-branch`
6. 在原项目上创建 Pull Request。

提示：本章小结

- 远程仓库是托管在服务器上的 Git 仓库副本，origin 是最常用的远程名
- `git push` 上传本地提交，`git pull` 下载并合并，`git fetch` 只下载不合并
- 在 GitHub 上可以创建仓库、管理远程仓库
- Pull Request 是 GitHub 的核心协作功能，用于代码审查和合并
- Fork 是贡献开源项目的标准方式

下一章预告：我们将深入了解 VS Code 中的 Git 集成，学习如何用图形界面高效操作 Git。

动手实验 5：推送到 GitHub 并创建 PR

花 5 分钟把本地分支推送到 GitHub，并在网页上创建一次 Pull Request。全程只使用鼠标点击，首次推送时按浏览器提示完成登录即可。

1. 点击底部状态栏分支名 → 「Create new branch…」，输入 `feature/pr-demo` 并回车（从 `main` 创建新分支）。
2. 点击资源管理器「新建文件」图标创建 `README.md`，输入我的第一个 GitHub 仓库，保存；点击文件旁「+」暂存，输入提交信息添加 `README`，点击「提交」。
3. 点击源代码管理视图顶部的「…」菜单 → 「推送」（首次会显示「发布分支」），点击「发布分支」。
4. 在弹出菜单中选择「登录到 GitHub」，按提示在浏览器中点击「Authorize」完成授权；随后选择「发布到 GitHub 公开仓库」或「发布到 GitHub 私有仓库」并确认。
5. 点击右下角通知中的「Open on GitHub」按钮，在仓库页面点击「Compare & pull request」，填写标题添加 `README`，点击「Create pull request」；再点击「Merge pull request」→ 「Confirm merge」完成合并。

提示：成功标志：GitHub 仓库页面的 `main` 分支中出现了 `README.md`，Pull Request 状态显示为 Merged（已合并）。

Chapter 6

VS Code 中的 Git 集成

这是本文档的核心章节。前面章节已经零散介绍了 VS Code 的 Git 功能，本章将系统性地展示所有功能，帮助你建立完整的认知。

6.1 源代码管理视图（Source Control）

为什么需要图形界面？

虽然 Git 命令行功能强大，但对于初学者来说，图形界面（GUI）有以下优势：

- **可视化**：直观地看到哪些文件被修改、哪些已暂存、哪些未跟踪
- **低门槛**：不需要记忆命令，通过点击按钮即可完成操作
- **Diff 对比**：内置强大的文件对比功能，一眼看出修改内容
- **冲突解决**：图形化界面让冲突解决变得简单直观
- **集成度高**：与编辑器深度集成，开发流程更流畅

VS Code 的 Git 集成是业界公认最出色的之一，足以满足日常 90% 的 Git 操作需求。

6.1.1 打开方式

- **侧边栏图标**：点击左侧活动栏中的第三个图标（分叉图标）。
- **快捷键**：Ctrl + Shift + G。
- **命令面板**：Ctrl + Shift + P → 输入”Source Control”。

6.1.2 界面详解

界面各区域如图 6.1 所示——

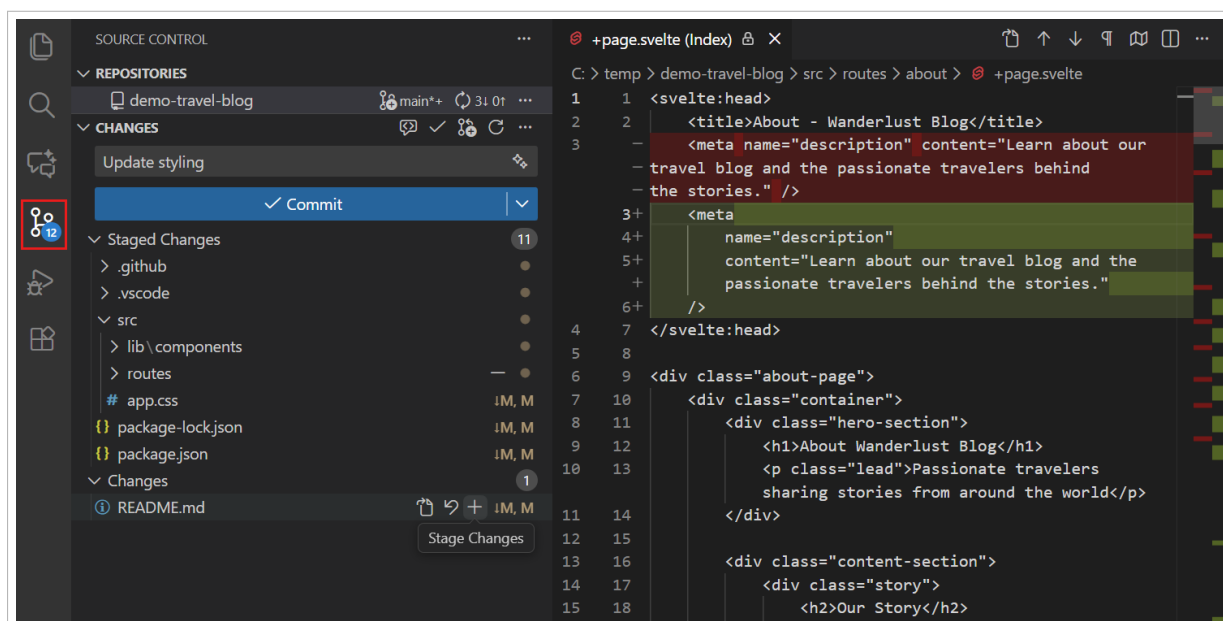


图 6.1: 源代码管理视图完整界面——各区域标注: (A) 提交信息输入框 (B) 提交按钮及下拉菜单 (C) 暂存的更改 (D) 更改列表 (E) 文件状态标记

界面各区域功能详解:

1. **提交信息输入框 (A):** 顶部的文本框, 用于输入提交信息。支持多行——第一行为标题, 空行后为详细描述。
2. **提交按钮 (B):** 点击提交暂存的变更。点击下拉箭头可选择”提交并推送”、”提交并同步”等。
3. **暂存的更改 (C):** 已暂存、等待提交的文件。只有这里的文件会被包含在下次提交中。
4. **更改 (D):** 已修改但未暂存的文件。
5. **文件状态标记 (E):**
 - U: Untracked (未跟踪) —— 新文件
 - A: Added (已暂存的新文件)
 - M: Modified (已修改)
 - D: Deleted (已删除)
 - R: Renamed (已重命名)
 - C: Copied (已复制)

6.1.3 提交历史图（Source Control Graph）

VS Code 新增了源代码管理图形视图，直观展示提交历史和分支关系。每个节点代表一次提交，绿色圆点表示当前分支的提交，点击可查看详细信息，如图 6.2 所示：

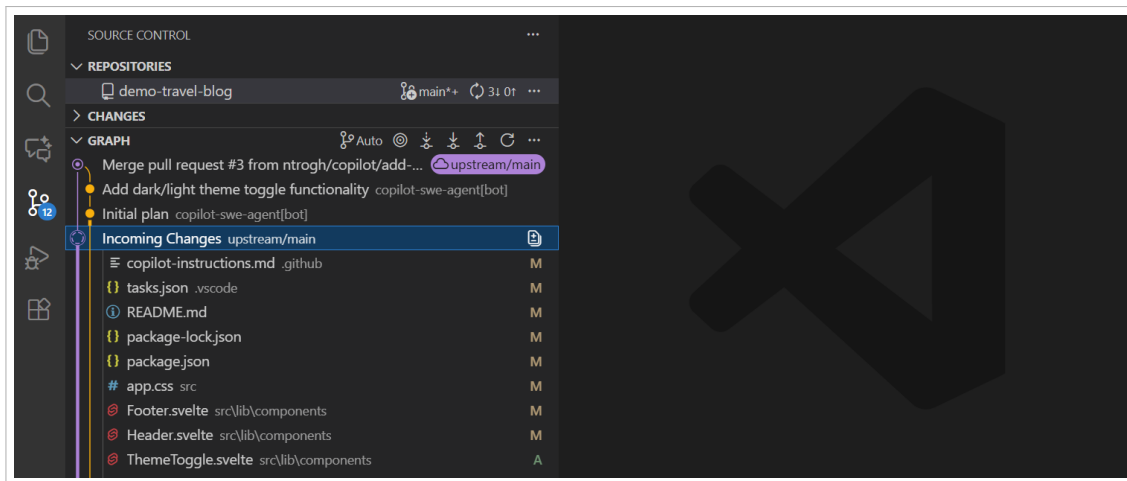


图 6.2: 源代码管理图形视图——以时间线形式展示提交历史，不同颜色区分不同分支，点击节点可查看详细信息

6.2 暂存（Stage）与提交（Commit）详解

6.2.1 暂存文件

暂存操作如图 6.3 所示——

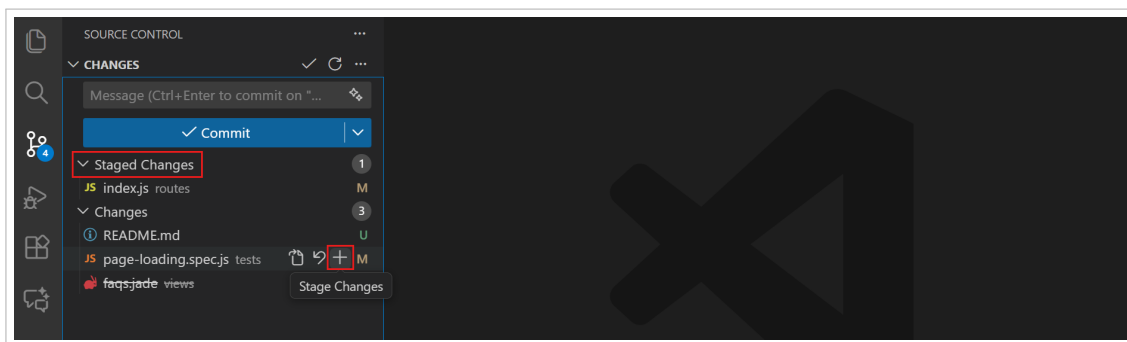


图 6.3: 暂存操作详情——每个文件右侧有“+”按钮可以单独暂存，“更改”标题旁有全部暂存按钮

6.2.2 部分暂存（Stage Selected Ranges）

VS Code 支持精细到代码行级别的暂存——一个文件中改了多处，可以只暂存其中一部分，如图 6.4 所示：

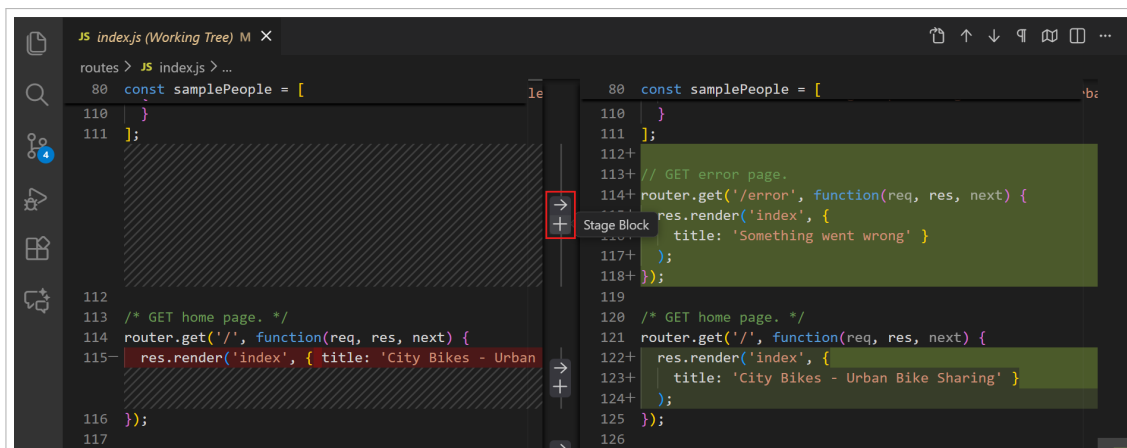


图 6.4: 部分暂存——打开文件 Diff 后，可以选择特定的代码块进行暂存，而非整个文件

部分暂存操作步骤:

1. 点击文件名打开 Diff 视图。
2. 选中你要暂存的代码行（或用鼠标拖选区域）。
3. 右键 → ”Stage Selected Ranges”。
4. 只有选中的代码块会被暂存，其余修改保留在”更改”中。

6.3 同步操作详解

6.3.1 一键同步

VS Code 底部状态栏显示同步状态，点击同步图标可以一键执行 pull + push，如图 6.5 所示：

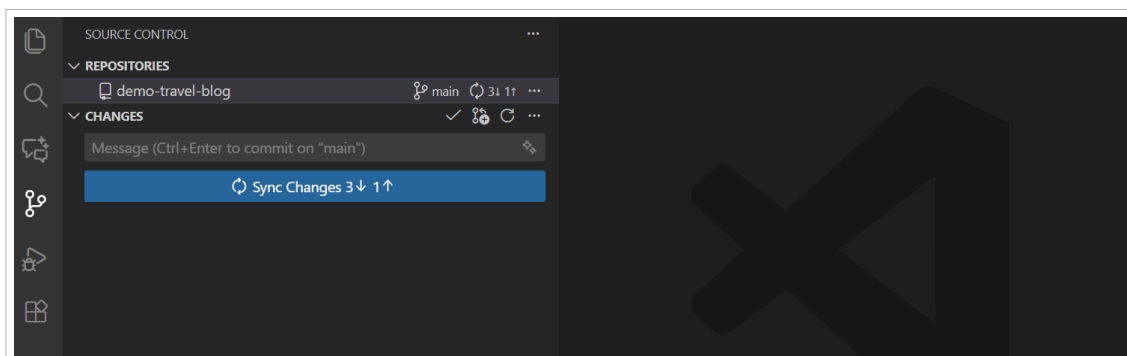


图 6.5: 同步更改——底部状态栏显示当前分支和同步状态（↑ 待推送，↓ 待拉取），点击旋转箭头图标执行一键同步

6.4 分支管理详解

6.4.1 查看和切换分支

当前分支名称显示在 VS Code 底部状态栏的左下角，点击即可查看分支列表、切换分支或创建新分支。创建新分支时，VS Code 会自动从当前分支创建并立即切换，如图 6.6 所示：

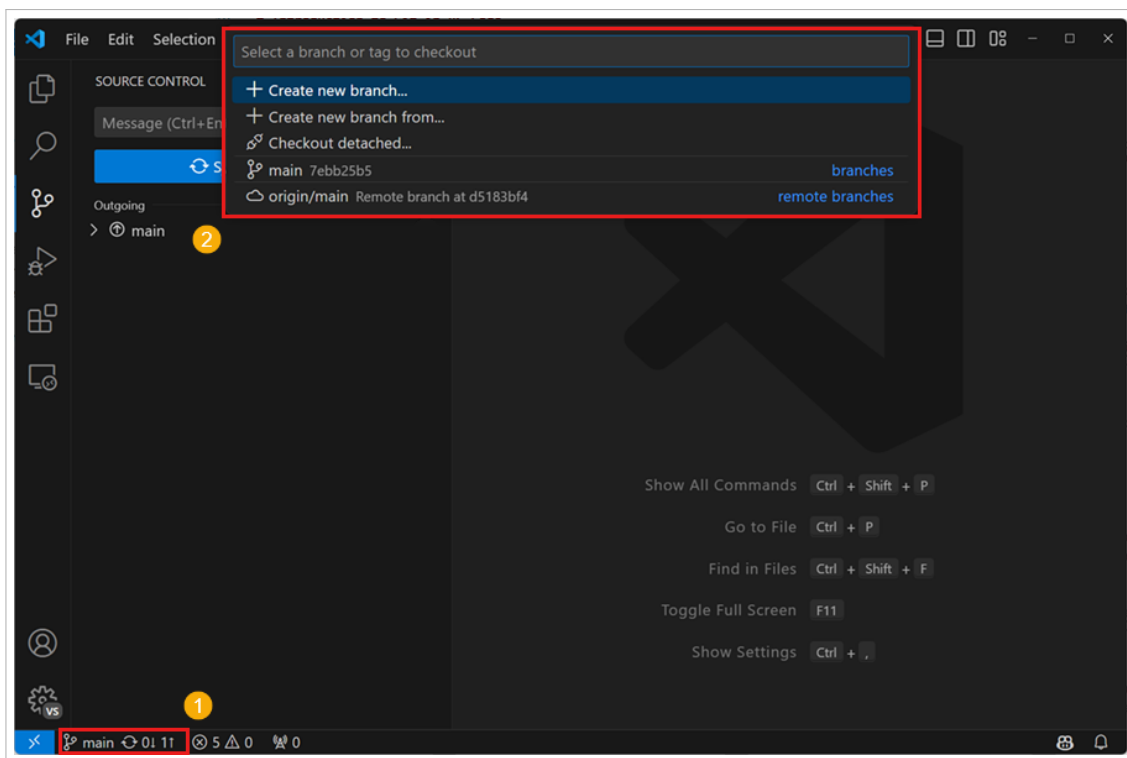


图 6.6: 创建新分支——输入新分支名称后，VS Code 会自动从当前分支创建并切换

6.5 Timeline 视图：文件历史

VS Code 的 Timeline 视图（默认在资源管理器底部）显示当前文件的所有提交历史，如图 6.7 所示：

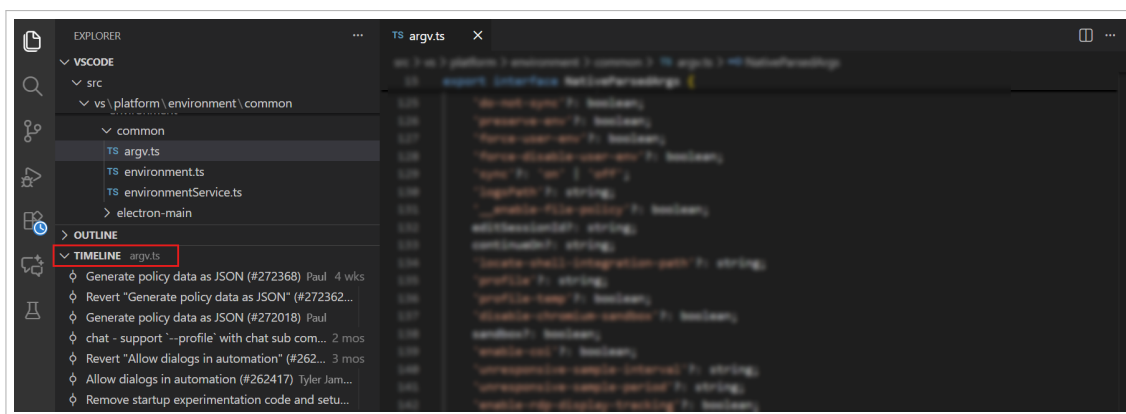


图 6.7: Timeline 视图——显示当前文件的所有提交历史，按时间排列。点击任意提交可查看该次修改的详细内容，右键可选择”比较”或”还原”

6.6 Stash（暂存工作）

当你需要切换分支但当前有未完成的修改时，Stash 功能可以临时保存你的工作：

1. 打开命令面板（Ctrl + Shift + P）。
2. 输入”stash”，选择相应操作：
 - **Git: Stash:** 暂存当前修改（工作区恢复干净状态）。
 - **Git: Stash (Include Untracked):** 暂存包括未跟踪文件。
 - **Git: Stash Pop:** 恢复最近一次暂存并删除该 stash。
 - **Git: Stash Apply:** 恢复最近一次暂存但保留 stash。
 - **Git: Stash Drop:** 删除最近一次暂存。

6.7 Git Blame 与行内注释

安装 GitLens 扩展后，VS Code 会在每一行代码后面显示最后修改者和时间，如图 6.8 所示：

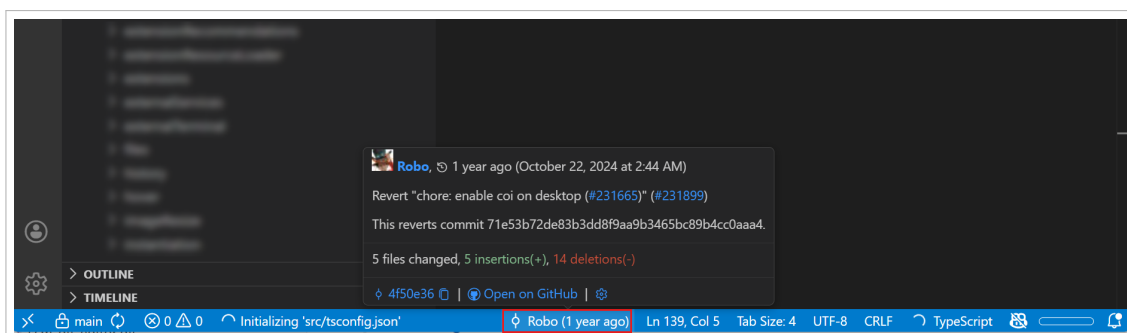


图 6.8: GitLens 的行内 Blame 注释——每行代码后显示灰色文字：最后修改者、时间和提交信息摘要

6.8 推荐扩展

以下扩展可以显著增强 VS Code 中的 Git 体验：（见表 6.1）

表 6.1: 推荐 VS Code 扩展

扩展名	作者	功能
GitLens	GitKraken	行内 blame 注释、提交历史可视化、分支比较等
GitHub Pull Requests	GitHub	在 VS Code 中查看、创建、审查 PR
GitHub Issues	GitHub	在 VS Code 中管理 GitHub Issues
Git Graph	mhutchie	图形化显示提交历史和分支图
Git History	Don Jayamanne	查看文件/行的详细提交历史

提示：强烈推荐安装 **GitLens**。它提供了：

- **行内 blame**：在每一行代码后面显示最后修改者和时间。
- **悬停详情**：鼠标悬停在代码上，显示该行的完整提交信息。
- **文件历史**：查看文件的完整修改时间线。
- **比较分支**：可视化比较任意两个分支的差异。

6.8.1 GitLens 详细使用指南

GitLens 是 VS Code 中最流行的 Git 增强扩展，它能在编辑器中直接显示代码的提交历史和作者信息，帮助初学者快速理解代码的演变过程。

安装步骤：按 `Ctrl+Shift+X` 打开 Extensions 面板，搜索 **GitLens**，点击 **Install**。

核心功能：

- **Inline Blame:** 安装后自动在每行代码末尾显示最后修改者和时间。
- **CodeLens:** 在函数和类上方显示作者和提交信息。
- **GitLens: Show Commit Graph:** 打开提交历史图形视图。
- **GitLens: Open File History:** 右键文件 → 选择此命令查看完整历史。
- **GitLens: Toggle Line Blame / GitLens: Toggle Git CodeLens:** 快速开关行内 blame 和 CodeLens 显示。



图 6.9: GitLens 行内 Blame 与 CodeLens 显示

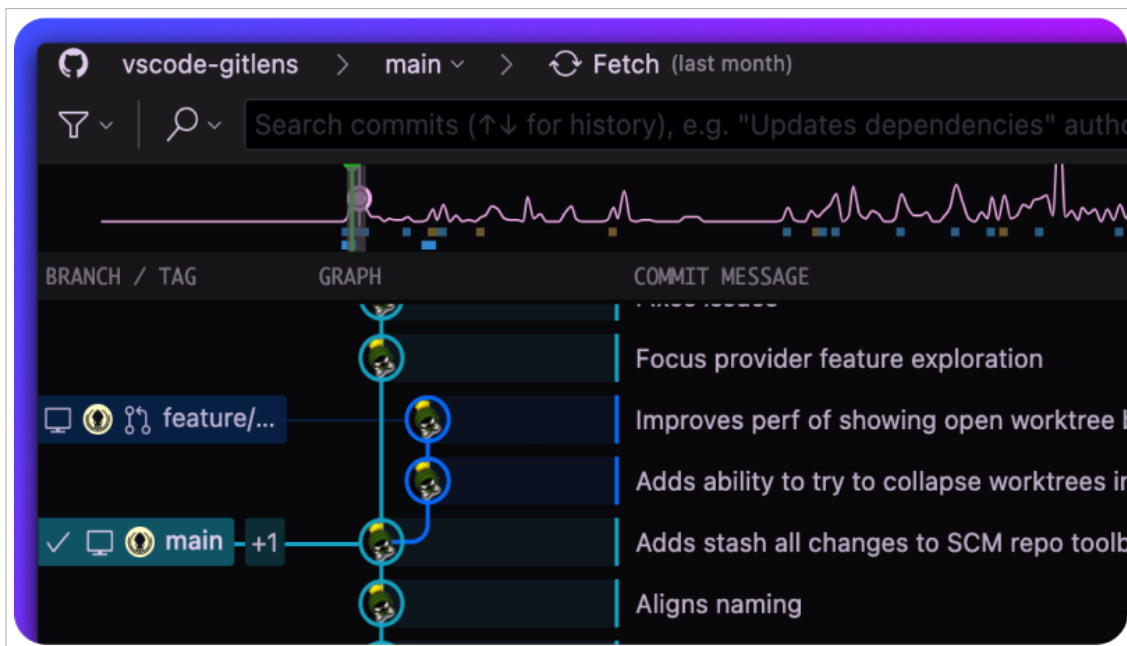


图 6.10: GitLens 提交图 (Commit Graph)

提示：初学者可以先开启 Inline Blame 和 CodeLens，它们能让你在阅读代码时自然了解修改背景。

6.8.2 GitHub Pull Requests 使用指南

GitHub Pull Requests 扩展让你在 VS Code 中直接查看、创建和审查 Pull Request，无需频繁切换浏览器。

安装步骤：按 `Ctrl+Shift+X` 打开 Extensions 面板，搜索 GitHub Pull Requests and Issues，点击 Install。安装后需要登录 GitHub：按 `Ctrl+Shift+P` 打开命令面板，输入 GitHub: Sign in 并按提示完成授权。登录后 Activity Bar 会出现 GitHub 图标。

核心功能：

- **Activity Bar 中的 GitHub 图标：**点击后查看当前仓库的 PR 列表。
- **GitHub Pull Requests: Create Pull Request：**从当前分支创建 PR。
- **查看 PR 详情：**点击 PR 列表中的条目，可查看 diff 和评论。
- **Checkout PR：**点击 Checkout 按钮切换到 PR 对应的分支进行审查。

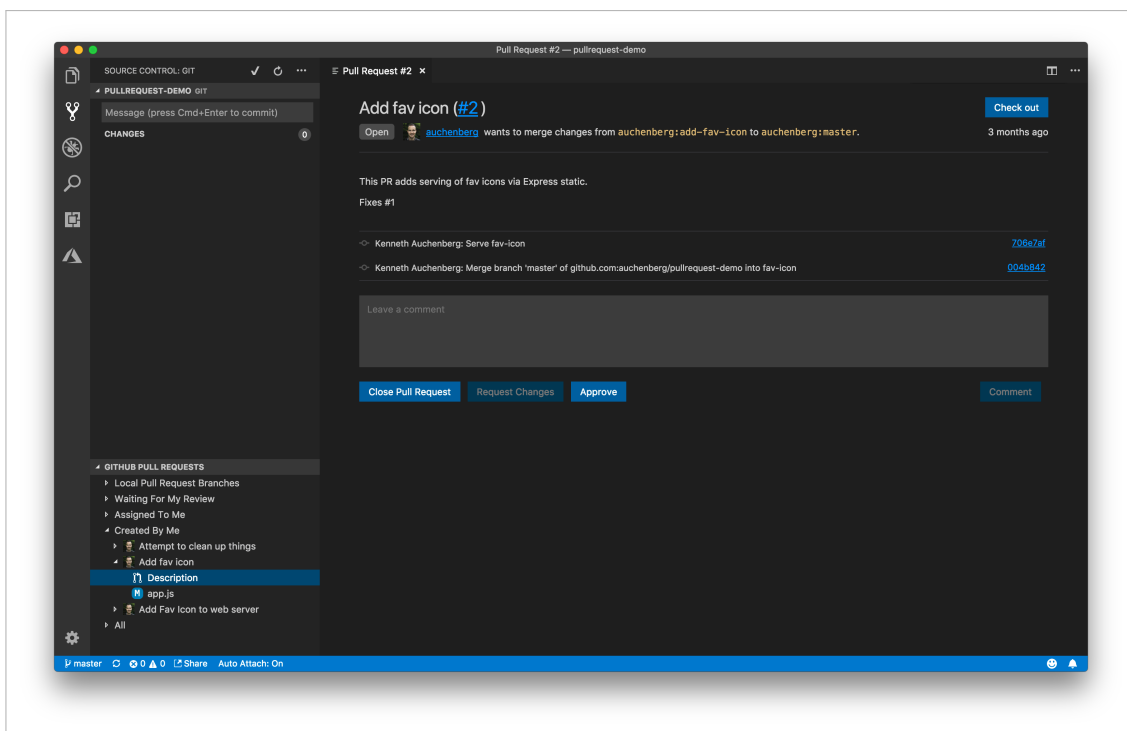


图 6.11: GitHub Pull Requests 侧边栏中的 PR 列表

提示：审查队友的 PR 时，可以直接在代码行上添加评论，所有操作都会同步到 GitHub。

6.8.3 GitHub Issues 使用指南

GitHub Issues 扩展将 Issue 跟踪功能集成到 VS Code 中，方便你在编码时随时查看和管理任务。

安装步骤：按 `Ctrl+Shift+X` 打开 Extensions 面板，搜索 GitHub Issues，点击 Install。

核心功能：

- **侧边栏 GitHub Issues 视图：**显示当前仓库的未关闭 Issues。
- **GitHub Issues: Create Issue from Selection:** 选中代码后快速创建 Issue。
- **引用关联：**在提交信息或 PR 描述中输入 `#issue-number` 自动关联对应 Issue。
- **创建新 Issue：**点击侧边栏中的按钮直接创建。

提示：与 GitHub Pull Requests 扩展配合使用，可以在一个界面中同时管理 Issues 和 PRs。

6.8.4 Git Graph 使用指南

Git Graph 提供直观的图形化分支时间线，让初学者一眼看清项目的分支结构和合并历史。

安装步骤：按 `Ctrl+Shift+X` 打开 Extensions 面板，搜索 Git Graph，点击 Install。

核心功能：

- Git Graph: View Git Graph: 打开彩色分支时间线视图。
- 右键点击提交：选择 Cherry Pick、Revert、Checkout、Create Branch 或 Merge。
- 右键点击分支：选择 Checkout、Delete、Merge、Push 或 Rebase。
- 比较提交：`Ctrl+Click` 第二个提交，比较任意两个提交的差异。
- 搜索提交：在图形视图中按 `Ctrl+F` 搜索提交。

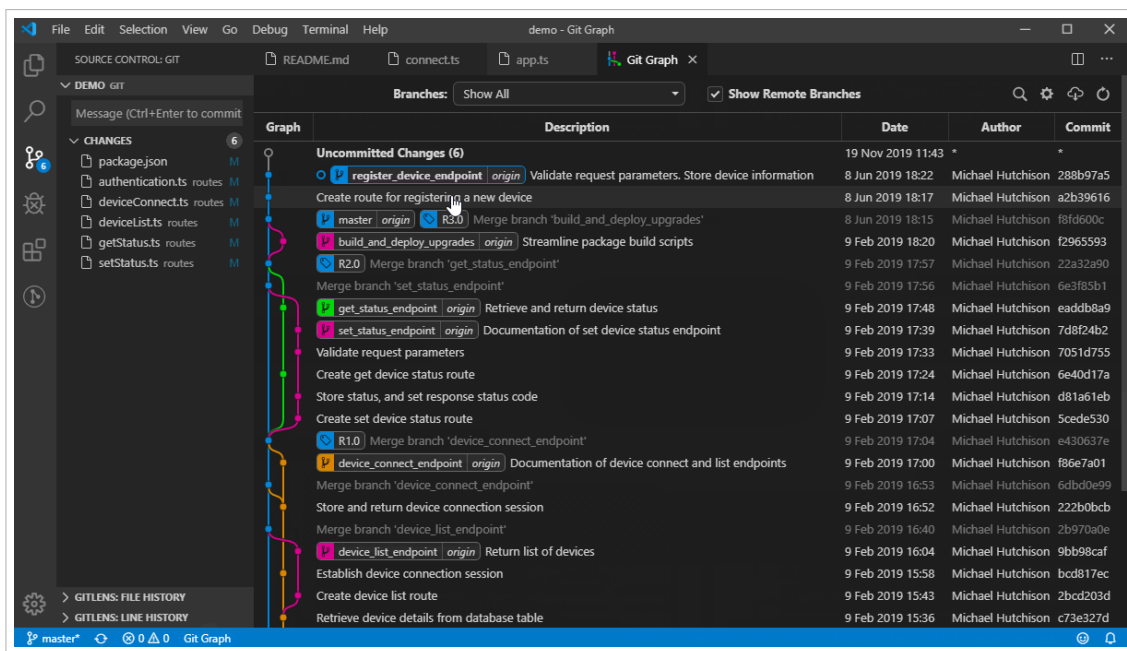


图 6.12: Git Graph 分支时间线视图

提示：学习分支与合并时，用 Git Graph 配合第四章内容，能直观理解分支的创建和合并过程。

6.8.5 Git History 使用指南

Git History 专注于查看单个文件或代码行的详细提交历史，界面轻量，适合快速追溯代码变更。

安装步骤：按 `Ctrl+Shift+X` 打开 Extensions 面板，搜索 Git History，点击 Install。

核心功能：

- Git: View File History: 查看当前文件的完整提交历史。
- Git: View Line History: 查看光标所在行的修改历史。
- Git: View Branch History: 查看当前分支的提交历史。
- Git: View Commit History: 查看整个仓库的提交历史。

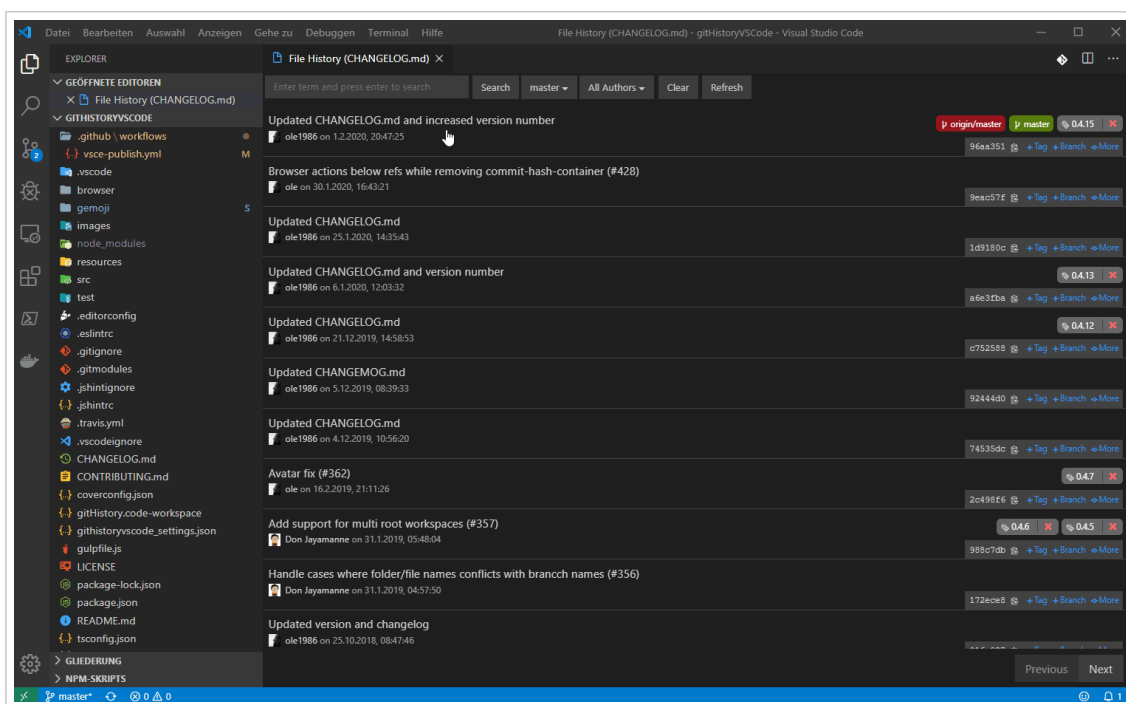


图 6.13: Git History 文件历史视图

提示：使用前先打开目标文件，再运行命令，否则可能看不到预期的历史记录。

动手实验 6：VS Code 中完成完整 Git 工作流

花 5 分钟把本章的核心功能串起来走一遍：建分支、编辑、部分暂存、提交、查看历史图。全程只使用鼠标点击，不需要输入任何命令。

1. 点击底部状态栏分支名 → 「Create new branch…」，输入 feature/sc-practice 并回车。
2. 在 hello.txt 末尾新增两行内容并保存；点击文件名打开 Diff 视图，用鼠标选中其中一行，右键 → 「Stage Selected Ranges」，实现只暂存这一行（部分暂存）。
3. 点击「更改」标题右侧的「+」按钮，把剩余修改全部暂存；在提交信息框输入练习部分暂存，点击「提交」。

4. 点击源代码管理视图标题栏的「提交历史图」图标,在图形视图中观察 `feature/sc-practice` 相对 `main` 的分叉和本次提交。
5. 在左侧资源管理器底部打开 Timeline（时间线）视图,点击 `hello.txt`, 可以看到该文件从实验 1 到现在的每一次提交记录。

提示：成功标志：Timeline 视图中 `hello.txt` 列出了多条提交记录,最近一条是练习部分暂存。

下一章预告：我们将学习如何将这些技能整合成一套完整的工作流,以及如何在团队中高效协作。

Chapter 7

实战 workflow

本章学习目标：

1. 掌握完整的 Feature Branch 开发流程
2. 学会在 VS Code 中执行完整的 Git 工作流
3. 了解多人协作中的最佳实践
4. 熟悉常见问题的解决方法

7.1 完整的 Feature Branch 开发流程

为什么需要规范的工作流？

想象一个团队有 5 个开发者，每个人都在同时开发不同的功能：

- 如果没有规范，大家都往 `main` 分支直接推送，代码会频繁冲突，项目随时可能崩溃
- 如果没有 PR 审查，有问题的代码可能直接上线，导致生产环境故障
- 如果没有分支策略，团队成员不知道谁在开发什么，沟通成本极高

Feature Branch 工作流通过以下原则解决这些问题：

1. 主分支稳定： `main` 分支始终是可部署的
2. 功能隔离： 每个功能在独立分支开发，互不影响
3. 代码审查： 通过 PR 确保代码质量

4. 历史清晰：合并后的历史记录完整可追溯

以下是一个完整的功能开发流程示例，从创建分支到合并：

```
===== 第一步：确保 main 分支是最新的 =====
git switch main
git pull --rebase origin main

===== 第二步：创建功能分支 =====
git switch -c feature/user-auth

===== 第三步：开发并提交（多次小提交） =====
... 编写代码 ...
git add src/auth/login.ts
git commit -m "添加登录表单组件"

... 继续编写 ...
git add src/auth/auth-service.ts
git commit -m "实现 JWT 认证服务"

... 添加测试 ...
git add tests/auth.test.ts
git commit -m "添加认证模块单元测试"

===== 第四步：推送到 GitHub =====
git push -u origin feature/user-auth

===== 第五步：在 GitHub 上创建 Pull Request =====
gh pr create --title "添加用户认证功能" --body "实现了 JWT 登录"

===== 第六步：根据审查意见修改 =====
git add .
git commit -m "修复审查意见中的问题"
git push    PR 自动更新

===== 第七步：合并后清理 =====
git switch main
git pull
git branch -d feature/user-auth
git push origin --delete feature/user-auth
```


7.2 VS Code 中的完整操作流程

同样的流程，在 VS Code 中的操作步骤，如图 7.1 所示：

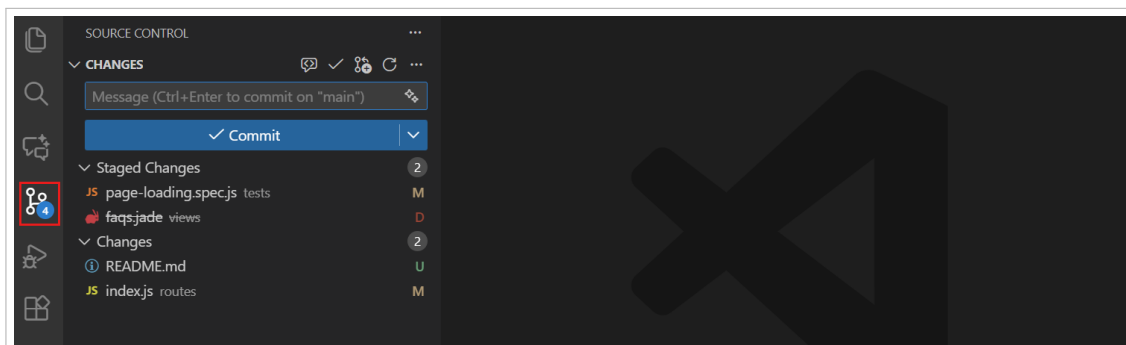


图 7.1: 开发中的 VS Code——编辑器 Gutter 实时显示你的修改（绿色/蓝色/红色标记），源代码管理视图列出所有变更文件

1. **同步**：点击底部状态栏的同步按钮，确保本地是最新的。
2. **创建分支**：点击底部状态栏的分支名称 → "Create new branch" → 输入 `feature/user-auth`。
3. **开发**：编写代码。编辑器中的 gutter 会实时显示变更。
4. **暂存**：在源代码管理视图中，点击文件旁的 "+" 暂存修改。对于大文件，可以打开 Diff 视图后部分暂存。
5. **提交**：输入提交信息，点击"提交"。
6. **推送**：点击"..." → Push，或点击提交按钮旁的下拉箭头选择"提交并推送"。
7. **创建 PR**：源代码管理视图会出现"Create Pull Request"按钮，点击即可。
8. **审查**：安装 GitHub Pull Requests 扩展后，可以直接在 VS Code 中审查 PR。
9. **合并**：在 GitHub 网页上点击"Merge"，或扩展中操作。
10. **清理**：使用命令面板 `Git: Delete Branch` 删除已合并的分支。

7.3 日常开发的最佳实践

日常工作习惯：

1. **开始工作前**：先 `git pull --rebase` 同步远程更新。

2. 创建功能分支：永远不要在 `main` 上直接开发。
3. 频繁提交：每个提交只做一件事，保持提交粒度小且清晰。
4. 提交前检查：用 `git diff` 或 VS Code Diff 视图确认修改内容。
5. 写清楚提交信息：遵循“祈使语气 + 50 字符以内”的规则。
6. 推送前同步：推送前先 `pull`，避免推送失败。
7. 合并后清理：删除已合并的功能分支，保持仓库整洁。

⚠ 注意

常见错误与预防：

- 提交了不该提交的文件：养成先配置 `.gitignore` 再开始开发的习惯。
- 在 `main` 上直接开发：始终使用功能分支。
- 提交信息模糊：写“fix”不如不写。
- 长时间不推送：本地积累太多提交，增加冲突风险。
- 强制推送 `main`：永远不要 `git push --force` 到 `main` 分支。

7.4 常见问题 FAQ

Q1：我不小心提交了密钥文件，怎么办？

首先，立即更换泄露的密钥！然后使用以下命令从历史中彻底删除：

安装 `filter-repo` 工具（如果尚未安装）

```
pip install git-filter-repo
```

从所有历史中删除 `secret.txt`

```
git filter-repo --path secret.txt --invert-paths
```

强制推送到远程（这会改写历史！）

```
git push origin --force --all
```

注意：这会改写提交历史，如果团队有其他人在协作，需要通知他们重新克隆仓库。

Q2：我删除了一个文件，还能恢复吗？

查看文件历史，找到删除前的最后一次提交

```
git log --diff-filter=D --summary
```

恢复文件到工作区

```
git restore --source=HEAD~1 -- filename.txt
```

或者从历史中恢复

```
git checkout HEAD~1 -- filename.txt
```

Q3: 如何撤销已经 push 的提交?

方法 1: 使用 revert (推荐, 不会改写历史)

```
git revert abc123
```

```
git push
```

方法 2: 使用 reset (会改写历史, 谨慎使用)

```
git reset --hard HEAD~1
```

```
git push --force-with-lease
```

Q4: 如何处理”Permission denied (publickey)” 错误?

这通常是因为 SSH 密钥配置问题:

检查 SSH 密钥是否存在

```
ls ~/.ssh/id_ed25519.pub
```

检查 SSH agent 是否运行

```
ssh-add -l
```

重新添加密钥

```
ssh-add ~/.ssh/id_ed25519
```

测试连接

```
ssh -T git@github.com
```

提示: 本章小结

- 完整的 Feature Branch 工作流: 同步 → 创建分支 → 开发 → 推送 → PR → 合并 → 清理
- VS Code 提供了完整的图形化操作界面, 无需记住命令行

- 日常开发最佳实践：小步提交、写清楚提交信息、先 pull 再 push
- 遇到问题不要慌，`git reflog` 可以帮你找回丢失的提交

附录预告：接下来是常用命令速查表，方便日常查阅。

动手实验 7：端到端 Feature Branch 开发流程

花 5 分钟走一遍真实的 Feature Branch 全流程：同步 → 建分支 → 多次小提交 → 推送 → PR → 合并 → 清理。全程只使用鼠标点击。

1. 确认底部状态栏分支名为 `main`，点击底部状态栏的「同步更改」旋转箭头图标拉取最新（若提示没有远程仓库，说明仓库尚未发布，直接跳到第 2 步）。
2. 点击底部状态栏分支名 → 「Create new branch…」，输入 `feature/e2e-flow` 并回车。
3. 新建文件 `e2e.md`，输入第一行内容保存，暂存并提交（提交信息添加项目说明）；再次追加一行内容，暂存并提交（提交信息补充使用说明），体验「小步提交」。
4. 点击「…」菜单 → 「推送」，将 `feature/e2e-flow` 推送到 GitHub（若未发布过仓库，按提示登录并在公开/私有仓库中选择一个）。
5. 点击通知中的「Open on GitHub」，点击「Compare & pull request」创建并合并 PR；回到 VS Code 点击「同步更改」，再点击「…」菜单 → 「分支」→ 「删除分支…」选择 `feature/e2e-flow`，完成清理。

提示：成功标志：GitHub 上 `main` 分支包含 `e2e.md`，且本地 `feature/e2e-flow` 分支已删除，整个 Feature Branch 流程闭环完成。

Chapter 8

新手避坑指南

本章收集了新手最容易踩的坑，按“场景—错误做法—正确做法—后果对比”四列呈现。建议先通读一遍建立印象，遇到对应场景再回来查阅细节。完整对照见表 8.1。

8.1 核心避坑表

表 8.1: 新手避坑速查表

场景	错误做法	正确做法	后果对比
提交了密钥/密码	直接再提交一次删除文件	1. 立即换密钥 2. <code>git filter-repo --path secret.txt --invert-paths</code> 3. <code>git push --force-with-lease</code>	历史仍可被挖掘 vs 彻底清除
误删文件想恢复	重新手写代码	<code>git restore --source=HEAD 1 -- filename.txt</code>	耗时易错 vs 秒级恢复
想撤销已推送的提交	<code>git reset --hard</code> + 强推	<code>git revert <commit-hash></code> + 推送	破坏他人历史 vs 安全新增反向提交

场景	错误做法	正确做法	后果对比
长期不拉取直接开发	直接在本地改代码	先 <code>git pull --rebase</code> 再开发	巨大冲突 vs 微小冲突
在 main 分支直接开发	不创建分支直接改	<code>git switch -c feature/xxx</code> 再开发	污染主线、难以回滚 vs 隔离实验、随时删除
解决冲突时手动删标记	手动编辑 <<<<<<< 标记	用 VS Code 三方合并编辑器点按钮	易漏改、逻辑错 vs 可视化、AI 辅助
提交信息写"fix" "update"	模糊提交信息	祈使语气 + 50 字内：修复登录页空指针	无法追溯 vs 一眼知意
强制推送 main 分支	<code>git push --force origin main</code>	绝不强推 main；用 <code>revert</code>	团队历史丢失 vs 历史完整
忽略.gitignore配置	先开发后配置	新建仓库第一步配.gitignore	提交垃圾文件、泄露密钥 vs 干净历史
合并冲突后忘记暂存	直接提交	冲突解决后 <code>git add .</code> 再提交	提交失败、冲突标记残留 vs 正常合并

8.2 紧急救命命令速查

几乎所有“搞砸了”都能用这两条命令救回来：

- 1. `git reflog` —— 查看 HEAD 所有移动记录，找到想回去的那一刻的哈希值
- 2. `git reset --hard < 哈希值 >` —— 直接穿越回那个时刻（慎用，会丢失当前未提交的改动）

实战演练：误删提交后恢复

- 1. 运行 `git reflog`，找到误删前的那一行（如 `HEAD@2: commit: 添加登录功能`）
- 2. 复制该行开头的哈希值（如 `a1b2c3d`）
- 3. 运行 `git reset --hard a1b2c3d`
- 4. 确认代码恢复，如已推送需 `git push --force-with-lease`

8.3 常见报错与一键修复

表 8.2: 常见报错与一键修复

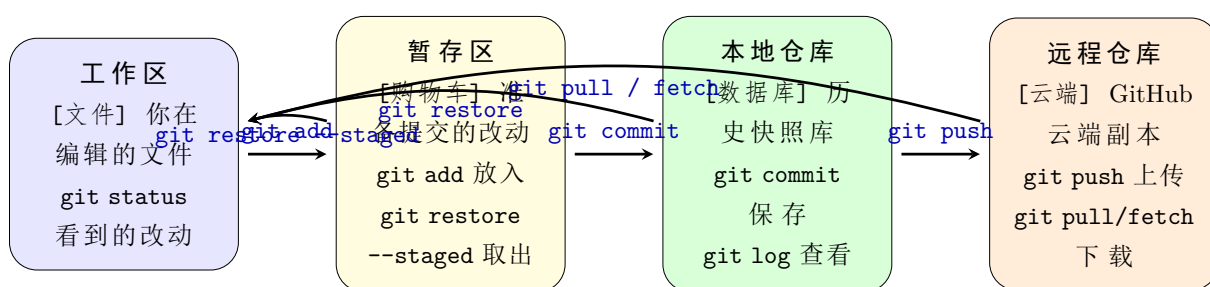
报错信息		原因与修复
Permission denied (publickey)		SSH 密钥未加入 agent 或 GitHub 未配置公钥。修复: <code>ssh-add ~/.ssh/id_ed25519</code> 并检查 GitHub Settings → SSH Keys
remote: Repository not found		远程地址错误或无权限。修复: <code>git remote -v</code> 核对 URL, 或重新 <code>git remote set-url origin < 正确 URL ></code>
fatal: refusing to merge unrelated histories		两个仓库无共同祖先。修复: <code>git pull origin main --allow-unrelated-histories</code>
Your branch is ahead of 'origin/main' by N commits		本地有未推送提交。修复: <code>git push</code>
fatal: not a git repository		当前目录非 Git 仓库。修复: <code>cd < 正确目录 ></code> 或 <code>git init</code>

可视化速查卡

本章提供三张全页核心流程图，建议打印贴在显示器旁，随时对照。

8.4 速查卡 1：Git 四大区域数据流转

完整数据流转如图 8.1 所示——



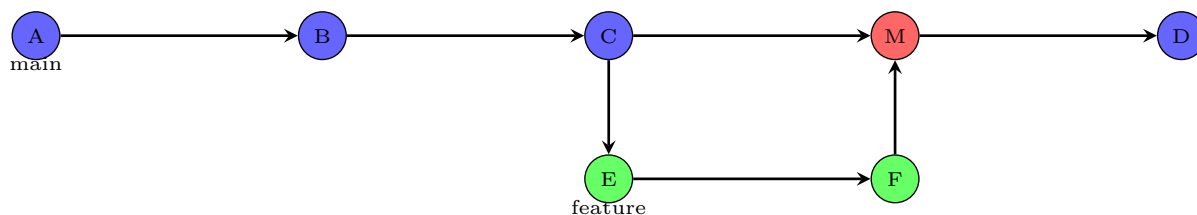
→ 正向操作（创建历史） ← 逆向操作（撤销/同步）
△ `git reset --hard` 危险：直接丢弃改动

图 8.1: Git 四大区域数据流转速查——工作区、暂存区、本地仓库、远程仓库之间的完整操作路径

8.5 速查卡 2：分支演变——合并 vs 变基

分支演变对比如图 8.2 所示——

Merge: 保留完整历史，产生菱形合并节点



Rebase: 线性历史，无合并节点

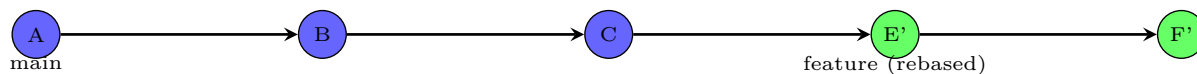


图 8.2: 分支演变对比速查——Merge 产生菱形合并节点，Rebase 生成线性历史

[分支] `git merge feature`: 保留分支上下文，适合团队公共分支

[魔法] `git rebase main`: 整理成直线，适合个人功能分支（推送前）

8.6 速查卡 3：远程协作完整链路

完整协作链路如图 8.3 所示——

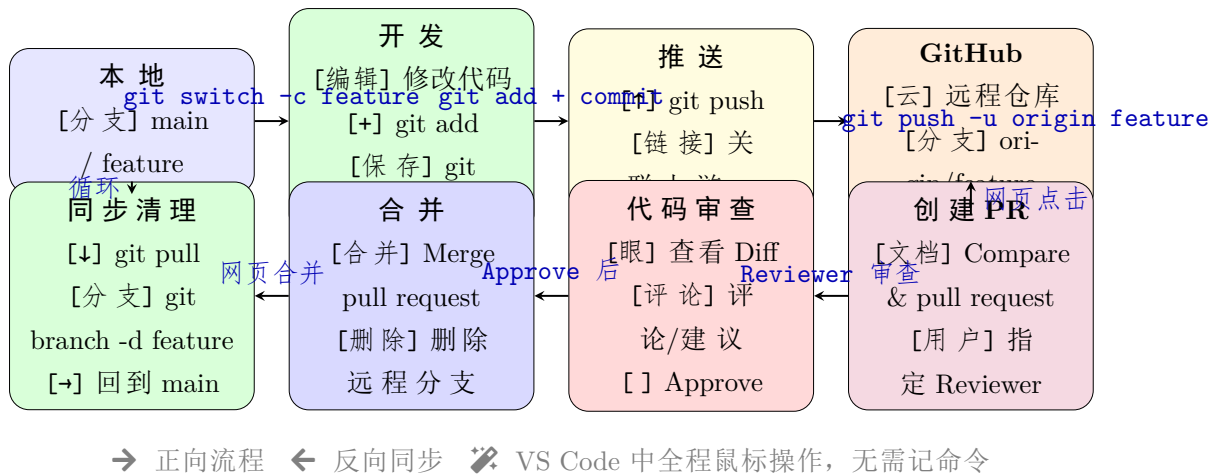


图 8.3: 远程协作完整链路速查——从本地开发到 GitHub 合并的完整流程

Chapter 9

附录：常用命令速查表

9.1 基础操作

表 9.1: 基础操作命令速查

命令	说明
<code>git init</code>	初始化新的 Git 仓库
<code>git clone <url></code>	克隆远程仓库
<code>git status</code>	查看工作区状态
<code>git status -s</code>	简洁状态输出
<code>git add <file></code>	暂存文件
<code>git add .</code>	暂存所有变更
<code>git commit -m "msg"</code>	提交暂存的变更
<code>git commit -am "msg"</code>	提交所有已跟踪文件的修改
<code>git log</code>	查看提交历史
<code>git log --oneline --graph</code>	图形化简洁历史
<code>git diff</code>	查看未暂存的修改
<code>git diff --cached</code>	查看已暂存的修改

9.2 分支操作

表 9.2: 分支操作命令速查

命令	说明
<code>git branch</code>	列出本地分支
<code>git branch <name></code>	创建新分支
<code>git switch <branch></code>	切换到分支
<code>git switch -c <branch></code>	创建并切换到新分支
<code>git branch -d <branch></code>	删除已合并的分支
<code>git merge <branch></code>	合并指定分支到当前分支
<code>git merge --abort</code>	取消正在进行的合并

9.3 远程操作

表 9.3: 远程仓库操作命令速查

命令	说明
<code>git remote -v</code>	查看远程仓库
<code>git remote add origin <url></code>	添加远程仓库
<code>git fetch origin</code>	下载远程更新（不合并）
<code>git pull</code>	拉取并合并远程更新
<code>git pull --rebase</code>	拉取并以 rebase 方式合并
<code>git push</code>	推送本地提交到远程
<code>git push -u origin <branch></code>	推送并设置上游追踪

9.4 撤销操作

表 9.4: 撤销与回退命令速查

命令	说明
<code>git restore <file></code>	撤销工作区的修改
<code>git restore --staged <file></code>	取消暂存

命令	说明
git commit --amend	修改上一次提交
git reset --soft HEAD~1	回退提交，保留修改在暂存区
git reset --hard HEAD~1	回退提交，丢弃所有修改
git revert <commit>	创建新提交来撤销指定提交
git reflog	查看 HEAD 移动历史（救命命令）

9.5 VS Code 快捷键速查

表 9.5: VS Code 常用快捷键速查

快捷键	命令	说明
Ctrl+Shift+G	Source Control	打开源代码管理视图
Ctrl+Shift+P	Command Palette	命令面板
Ctrl+Enter	Commit	提交
Alt+F5	Next Change	跳转到下一个变更

 学习建议

1. 动手实践：创建一个测试仓库，把本文档中的每个命令都执行一遍。
2. 先理解概念：工作区 → 暂存区 → 仓库 → 远程，理解数据流向。
3. 养成习惯：每次操作前先 `git status`，提交前检查 `git diff`。
4. 小步提交：每个提交只做一件事，写清楚提交信息。
5. 善用分支：每个功能一个分支，永远不要在 `main` 上直接开发。
6. 记住 `reflog`：几乎任何 Git 错误都可以通过 `git reflog` 恢复。
7. 持续学习：Git 功能丰富，先掌握核心 20%，再按需学习其余功能。

— 文档结束 —

本文档基于 Git 2.45+ 和 VS Code 2025 年最新版本编写。

截图来源：Visual Studio Code 官方文档（<https://code.visualstudio.com/docs>）

参考资料：<https://git-scm.com/doc>、<https://code.visualstudio.com/docs>